

Software Requirement Specification

For

Elderly Care Monitoring System

Version 1.0 approved

Prepared by Kathiravan A

Coimbatore institute of Technology

15.10.2026

Table of Contents

Table of Contents	i
Revision History	ii
1. Introduction	3
1.1 Purpose	3
1.2 Document Conventions	4
1.3 Intended Audience	4
1.4 Product Scope	4
1.5 References.....	4
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions.....	5
2.3 User Classes and Characteristics	5
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	7
2.6 User Documentation	7
2.7 Assumptions and Dependencies	8
3. External Interface Requirements	10
3.1 User Interfaces.....	10
3.2 Hardware Interfaces.....	11
3.3 Software Interfaces.....	11
3.4 Communications Interfaces	11
4. System Features	11
4.1 User Grievance Submission Module.....	12
4.2 Admin Grievance Management.....	12
5. Other Nonfunctional Requirements	13
5.1 Performance Requirements	13
5.2 Safety Requirements.....	14
5.3 Security Requirements.....	15
5.4 Software Quality Attributes.....	16
5.5 Business Rules.....	17
6. Other Requirements	17
6.1 Security Requirements.....	17
6.2 Software Quality Attributes.....	17
6.3 Business Rules.....	17
Appendix A: Glossary	17
Appendix B: Analysis Models	18

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this document is to present a detailed Software Requirements Specification (SRS) for the application software layer of the **Elderly Care Monitoring System (ECMS)**. This project focuses exclusively on the development of the web portal, mobile application, and backend cloud engine designed to ingest, process, and display health and location data. The software provides an interactive dashboard interface and an automated emergency notification pipeline, allowing caregivers, family members, and medical personnel to digitally monitor a senior's well-being and orchestrate swift responses to system-generated alerts.

1.2 Document Conventions

The Elderly Care Monitoring System (ECMS) is a software-based application consisting of a mobile app and a web portal designed to help families and medical staff monitor senior citizens. The software acts as a central hub that receives health data (like heart rate and blood oxygen levels) and location data sent from a senior's wearable tracker. The software automatically processes this incoming data to spot emergencies, like a sudden drop in vitals or a senior wandering away from a present safe zone, and displays everything on an interactive dashboard.

The application also includes these key features:

Role-Based Web Portal: Secure login screens that give family members, doctors, and care staff different levels of access to the senior's records.

Automated Emergency Dispatcher: A backend notification service that instantly sends push notifications, SMS texts, and automated email alerts to contacts when an emergency is triggered.

False Alarm Safeguard: A software-controlled countdown window that lets a senior tap their app or device screen to cancel an accidental emergency alert before notifications are sent out.

Interactive Health Dashboards: A user-friendly interface featuring clean charts and graphs that display historical trends for heart rate, daily steps, and sleep patterns over time.

1.3 Intended Audience

Developers: To understand the software architecture, REST/Web Socket API endpoints, backend data processing logic, and database schemas.

Project Supervisors / Evaluators: To review the functional software scope, data safety configurations, and architectural compliance with project guidelines.

End User: To understand how to navigate the web/mobile user interfaces, interpret dashboard analytics, and manage application configurations.

1.4 Product Scope

ECMS is a cloud-based software platform consisting of a responsive web dashboard and a mobile notification application. The system provides a centralized data ingestion layer that receives pre-collected health metrics (heart rate, SpO2, temperature) and geographic coordinates from external API streams. The core software engine analyzes incoming time-series data strings to algorithmically identify fall patterns or safety anomalies. It aims to minimize emergency communication delays by executing an automated, cloud-based notification pipeline that alerts designated application contacts via push notifications, email, and SMS.

1.5 References

IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.

HL7 (Health Level Seven) International – Standards for Electronic Health Information Exchange software architectures.

2. Overall Description

2.1 Product Perspective

ECMS is a standalone software application package that functions as an independent cloud data handler and user dashboard solution. It does not control physical tracking hardware directly; instead, it exposes secure API gateways designed to receive standardized JSON data payloads from external device servers.

The system architecture is divided into three distinct software layers:

1. **The Ingestion & Processing Layer:** A Fast-API/Node.js backend engine.
2. **The Data Storage Layer:** A dual-database system optimized for relational profiles and time-series analytics.
3. **The User Presentation Layer:** Cross-platform mobile apps and web application portals.

2.2 Product Functions

- **Real-Time Data Pipeline:** Continuous, low-latency processing of incoming stream payloads containing vital health parameters.
- **Software-Based Fall Recognition:** Algorithmic evaluation of incoming 3-axis motion data arrays to flag threshold exceptions representing a fall event.
- **Automated Alert Dispatcher:** Cloud-based execution script that immediately routes high-priority multi-channel messages (SMS, email, mobile push tokens) upon anomaly detection.
- **Data Visualization & Analytics Reports:** Database query aggregation that renders historical health inputs into analytical tables, lines, and bar charts.

2.3 User Classes and Characteristics

Elderly Patients (Passive Data Models): Non-technical entities. They have no configured software user interfaces and do not log into the system. Their digital database profile acts exclusively as a passive data stream generator that receives, formats, and stores vital parameter strings.

Doctors / Medical Personnel (Web Dashboard Users): High clinical proficiency and moderate technical expertise. They log securely into the central web portal to view real-time patient health graphs, customize individual vital threshold boundaries, review systemic medical alarms, and manage the **Incident Action Tracker** to input clinical resolution logs.

System Administrators (IT & Institutional Staff): High technical and systems proficiency. They utilize an administrative backend portal to oversee global database operations, view systemic performance logs, assign doctor-to-patient access mappings via Role-Based Access Control (RBAC), and configure third-party notification API gateway keys (SMS/SMTP)

Use case Diagram:

Fig 2.1-Doctors

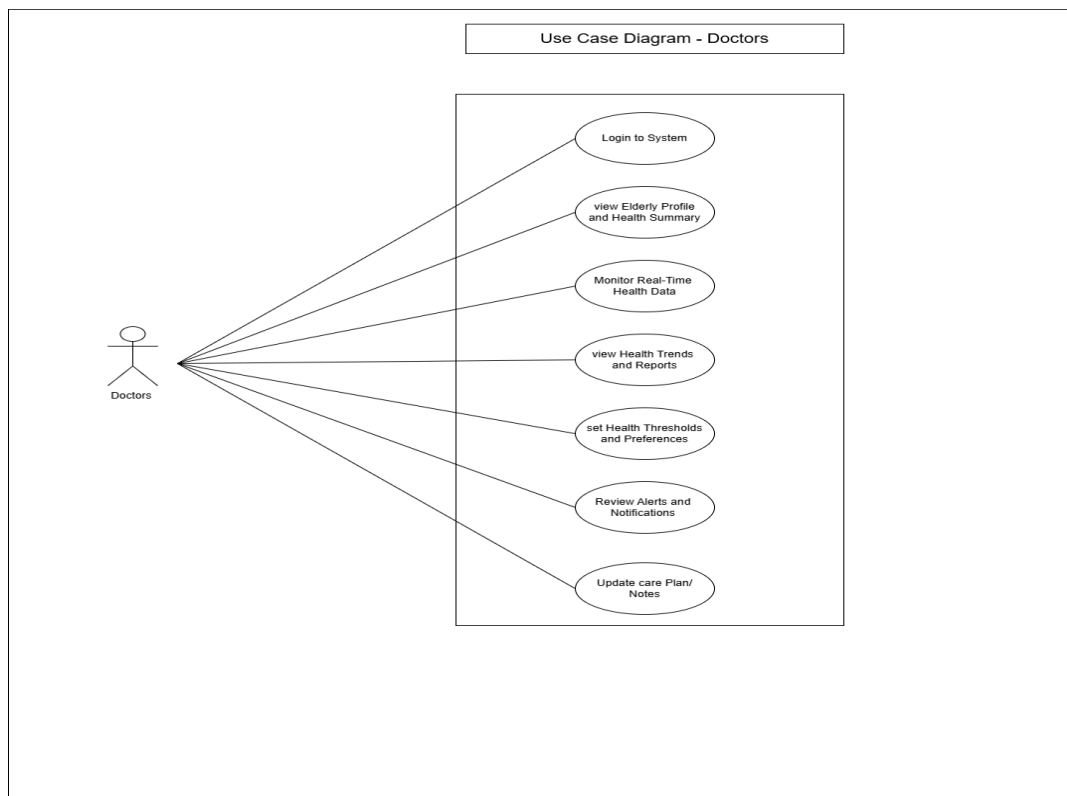


Fig 2.2 System Administrator

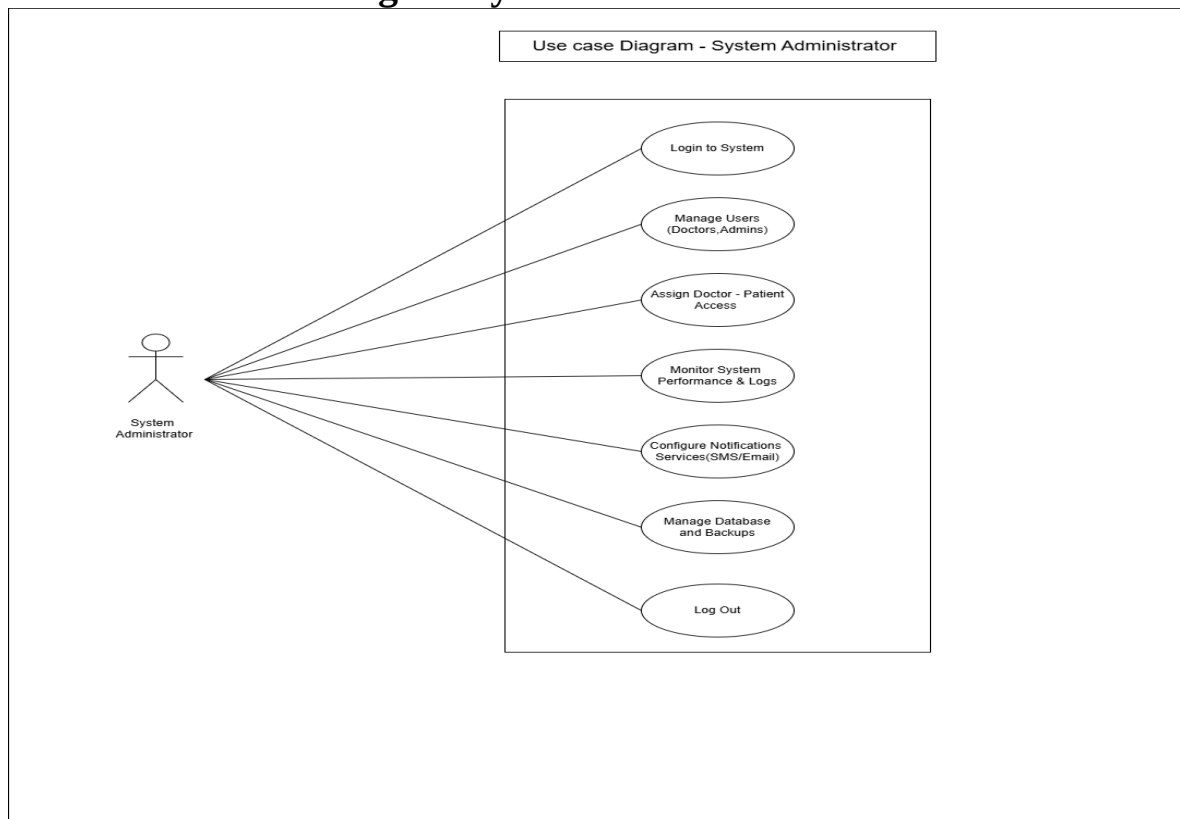
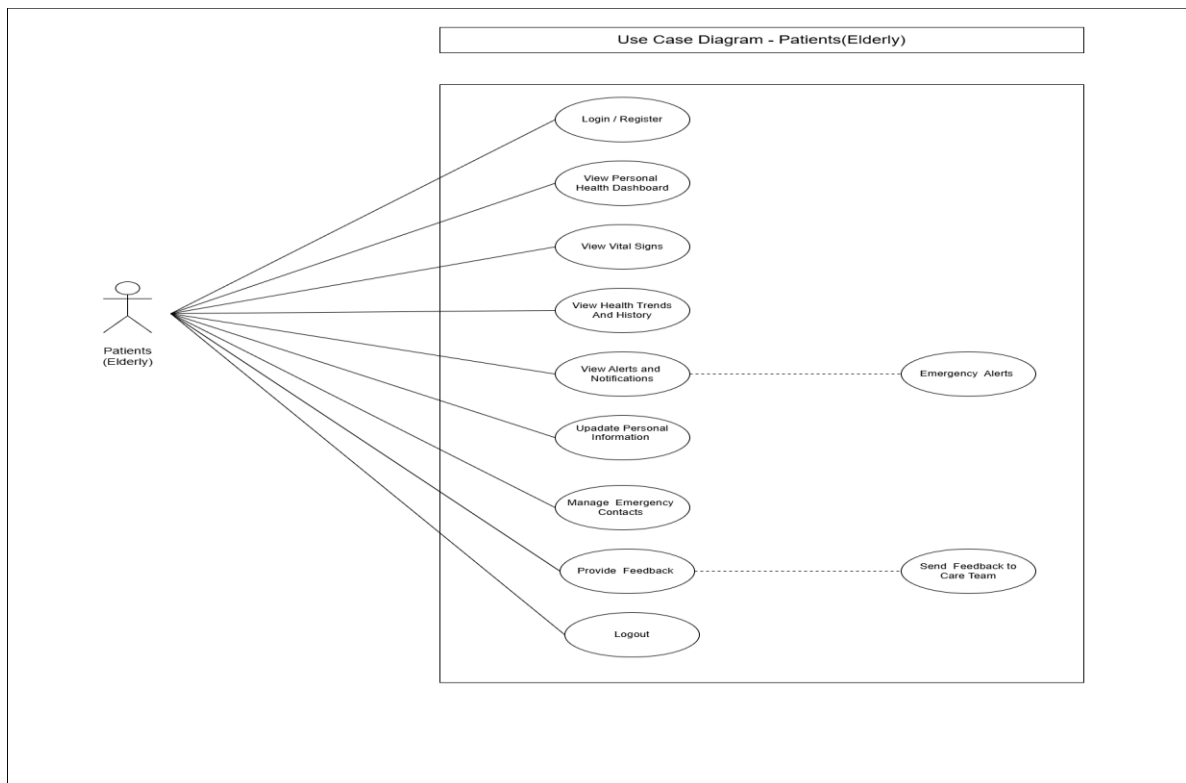


Fig 2.3 Patients



2.4 Operating Environment

- **Frontend Application Layer:** React.js (Web Portal Interface) and Flutter Framework (for cross-platform iOS and Android mobile software deployment).
- **Backend Application Server:** Fast API (Python) or Node.js runtime environment enforcing asynchronous request handling.
- **Database Management System:** PostgreSQL 15+ for core relational entities (user accounts, permissions, configuration profiles) integrated with Time ScaleDB/InfluxDB plugins for high-speed time-series sensor data indexing.
- **Deployment Infrastructure:** Linux-based virtual environments (Ubuntu Server OS hosted on cloud engines like AWS EC2 or Microsoft Azure).

2.5 Design and Implementation Constraints

- **Protocol Ingestion Limits:** The backend application layer must be structurally built to handle persistent, asynchronous Web-Socket or MQTT software listener connections without losing data packets.
- **Regulatory Compliance Framework:** The database access layers, session handling protocols, and encryption schemas must strictly comply with software security standards such as HIPAA or local data protection frameworks.
- **Mobile Resource Management:** The mobile application codebase must be explicitly optimized for background daemon execution, ensuring alert notification listeners remain fully functional even when the operating system places the app into low-power background restrictions.

2.6 User Documentation

The system will provide a comprehensive suite of software-specific documentation to ensure smooth on boarding, deployment, and system operation across all user tiers:

- **Caregiver Portal Manual:** A detailed, step-by-step user guide explaining how to navigate the mobile and web dashboards, manage user profiles, interpret real-time vital charts, and plot digital geo fence boundaries on the map interface.

- **Incident Resolution Guide:** A workflow manual instructing care staff and emergency contacts how to acknowledge, comment on, and securely close system-generated alert incidents directly inside the application console.
- **System Administrator and IT Deployment Guide:** Technical documentation covering the backend installation environment, Postgres SQL database configuration, API endpoint routing parameters, and SMTP/SMS notification gateway configurations for technical support personnel.
- **Embedded Software Help Section:** Contextual support integrated directly into the web and mobile applications, featuring descriptive tooltips, error-handling hints, and a frequently asked questions (FAQ) portal.

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- **Constant Network Connectivity:** It is assumed that administrative staff and caregivers have uninterrupted access to a stable internet connection and a modern web browser to receive low-latency alert updates and monitor real-time data charts seamlessly.
- **Pre-formatted Incoming Payloads:** It is assumed that external data streams will continuously transmit JSON payloads that strictly align with the system's expected schema rules and designated API endpoints.
- **Operational Readiness:** It is assumed that the primary caregivers and institutional medical teams are properly trained in navigating the software interfaces and handling system-generated alerts.
- **Gateway Reliability:** It is assumed that third-party notification servers will maintain a valid and operational status sufficient to deliver automated text and email alerts without platform-side delays.

2.7.2 Dependencies

- **Database Stability:** The application's core functionality depends entirely on the successful deployment and continuous availability of the PostgreSQL database server to read and write health logs without deadlocks.
- **Notification Infrastructure API Access:** The system relies on active subscriptions and functional API access keys from external SMTP and

SMS gateways to successfully dispatch all automated emergency alerts.

- **Operating System Environment:** The application deployment relies on compatible configurations within a Linux-based host environment (specifically Ubuntu Server or modern cloud engines) to maintain persistent backend processes.
- **Regulatory Compliance Frameworks:** Future additions and structural data safety policies remain strictly dependent on modifications to regional digital privacy laws and medical data protection standards.

3. External Interface Requirements

3.1 User Interfaces

The ECMS portal will feature a clean, high-contrast, responsive user interface designed using modern web and mobile frontend technologies. It follows accessible UI/UX principles, prioritizing readability for elderly-adjacent interfaces and layout simplicity for caregivers. The system provides:

- **Caregiver/Family Dashboard:** Secure mobile and web screens to view active patient lists, live metric widgets, and historical graphs.
- **System/Institutional Admin Dashboard:** A comprehensive interface to manage profiles, alter threshold properties, and view active system alerts.
- **User Profile Configuration Forms:** Input sheets for creating elderly profiles, setting health thresholds, and configuring emergency contact tiers.

Consistent elements across all web and mobile views will include:

- A standardized, persistent navigation layout containing links to Live Monitoring, Historical Reports, Configuration, and Support channels.
- Uniform interactive elements such as clear action triggers (e.g., "Clear Alert", "Save Configurations", "Back to Dashboard").
- High-visibility emergency pop-ups accented in strict crimson formatting with audible warning triggers.
- Inline input verification notifications displayed near form inputs to eliminate formatting mistakes.

3.2 Hardware Interfaces

As a pure cloud software system, ECMS does not directly execute local hardware firmware operations. It accesses external data fields through standard computer and network devices over internet layer protocols:

- **Data Access Devices:** Accessible via standard desktop computing environments, tablets, and smartphones running modern client OS variants (Android/iOS/Windows/macOS).
- **Network Host Frameworks:** Cloud servers interface with standard internet gateway modules to receive incoming asynchronous data feeds. No custom physical hardware integration resides inside this software core, but its ingestion gateways maintain compatibility with standard web network traffic packets.

3.3 Software Interfaces

ECMS interfaces directly with the following software layers and micro-components:

- **Database Management Engine (Post-gres SQL with Time-Scale DB extension):** Used for managing relational user access identities alongside specialized data structures for high-velocity health stream histories.
- **Backend Application Architecture (FastAPI):** Core web engine built to host server-side processes, evaluate incoming values, and feed client APIs.
- **Database Management Client (pgAdmin 4):** Utilized by administrative engineers to perform secure system queries and database maintenance.
- **Host Environment Platform:** Configured for cross-platform execution on modern enterprise servers, specifically optimized for Ubuntu Server Linux deployments.
- **Client App Environments:** Fully compatible with modern web standard runtimes (Chrome, Firefox, Safari, Edge) alongside native mobile web views.

3.4 Communications Interfaces

ECMS relies on standardized secure communication protocols to link client user interfaces with backend API channels:

- **Application Networking Protocol:** Enforces strict HTTPS communication for standard user sessions, database edits, and analytical queries.
- **Asynchronous Ingestion Channels:** Employs persistent, TLS-encrypted Web sockets streaming links to capture high-frequency metric payloads without network overhead.
- **Data Serialization Format:** Implements structured JSON formatting for all frontend-to-backend data payloads and inter-service responses.
- **Notification Engines:** Integrates with third-party web gateways (such as Twilio API and secure SMTP relay engines) to immediately distribute real-time automated SMS and email warning payloads.

4. System Features

4.1 User Grievance Submission Module(Real-Time Vital Ingestion & Processing Module)

4.1.1 Description and Priority

This software feature allows the system's backend to actively ingest, parse, and monitor real-time health stream metrics (heart rate, blood oxygen levels, temperature) from external connection brokers. It is **High Priority**, serving as the core analytical driver of the monitoring system.

- *Benefit: 9, Penalty: 9, Cost: 5, Risk: 4*

4.1.2 Stimulus/Response Sequences

1. External data stream pushes an inbound JSON metric packet into the backend socket endpoint.
2. The ingestion software parses the incoming packet, validating user ID matching and parameter formats.
3. The server checks values against the user's customized threshold ranges.

4. The system logs the entry into the time-series data store and reflects the update on active caregiver dashboard charts.
5. *Anomaly Condition:* If values violate bounds, the system updates the profile state to "Warning" and pushes it to the Alert pipeline.

4.1.3 Functional Requirements

- **REQ-1:** The software shall authenticate data streams before allowing metric entry injection.
- **REQ-2:** The software shall handle concurrent asynchronous incoming streams without losing data packet order.
- **REQ-3:** The software shall validate incoming metrics to ensure all required fields (timestamp, heart rate, \$SpO_2\$) are structurally correct.
- **REQ-4:** The software shall store metric parameters in a normalized time-series data table mapped to a unique user configuration ID.
- **REQ-5:** The software shall instantly push updated metric data arrays to active frontend dashboard connections using Web Sockets.

4.2 Admin Grievance Management(Automated Fall Recognition and Alert Dispatcher)

4.2.1 Description and Priority

This software process evaluates incoming motion array data strings using rule-based threshold filters to detect sudden fall anomalies and execute multi-channel alerts. This is a **High Priority** safety requirement.

4.2.2 Stimulus/Response Sequences

1. Backend receives a high-impact deceleration vector payload from an assigned client ID stream.
2. The tracking system flags a potential fall state and initiates an internal 15-second software countdown timer.
3. *Condition A:* If the user clears the countdown via the application client, the event terminates and transitions back to normal.
4. *Condition B:* If the countdown expires without user cancellation, the software changes the profile state to "Emergency".
5. The dispatcher runs alert triggers to send SMS, emails, and push alerts to all designated emergency contact paths.

4.2.3 Functional Requirements

- **REQ-6:** The software shall trigger a local verification state immediately upon receiving threshold-breaking motion vectors.
- **REQ-7:** The system shall maintain an accurate, non-blocking internal countdown mechanism for verification handling.
- **REQ-8:** The system shall cancel the alert pipeline instantly if a validated cancellation command is received before the countdown expires.
- **REQ-9:** The software shall extract the latest GPS tracking coordinates from the data tables during an active alarm dispatch event.
- **REQ-10:** The software shall route critical notification payloads to external API wrappers (SMS/SMTP gateways) when an anomaly state is confirmed.

5. Other Non-functional Requirements

5.1 Performance Requirements

- The backend API engine shall process standard incoming metric data streams and update internal database records within 1 second of receipt.
- The web and mobile applications shall load dashboard view structures within 2 seconds under regular server transaction rates.
- The emergency notification processing subsystem shall deliver dispatch requests to third-party SMS/Push APIs within 1.5 seconds of alert confirmation.
- The software architecture shall manage a base configuration of at least 500 parallel incoming device streams without latency drops.
- System deployment targets must maintain a structural monthly app availability rate of 99.9%.

5.2 Safety Requirements

- The software application must build automated error-catching loops to prevent state corruption during database dropouts.
- The system shall implement a 15-second local validation delay before executing full alert dispatches to filter out false alarms.

- Explicit modal warnings and dual-factor confirmations shall be integrated before allowing the deletion of any elderly profiles or user historical records.
- The backend software shall include rate-limiting boundaries on incoming API streams to protect server execution layers from Denial of Service (DoS) instabilities.

5.3 Security Requirements

- All user authentication protocols must hash profile passwords utilizing secure industry schemes before writing to the database.
- The software architecture must enforce HTTPS transport layers and SSL/TLS socket wrappers for all data transfers to prevent eavesdropping.
- JSON Web Tokens (JWT) with strict timeout intervals must handle mobile session maintenance and API access authorization.
- Input sanitization routines must process all incoming query parameters and payload strings to prevent SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) issues.
- Secure system access logging protocols must write all administrative profile modifications to isolated file records for auditing.
- Compliance: The software architecture shall align with necessary health data privacy regulations.

5.4 Software Quality Attributes

- **Usability:** Caregiver dashboards must feature clean layouts, readable fonts, and an interface that allows operators to find patient status details with minimal navigation steps.
- **Maintainability:** The backend code structure must adopt modular architectures with clean layer isolation to simplify debugging.
- **Reliability:** The core engine must implement automatic reconnection logic for its database connections and MQTT/Web Socket listeners when brief dropped states occur.
- **Scalability:** The software framework must use asynchronous handlers to support easy scaling across multi-container cloud deployments when customer registration numbers grow.
- **Testability:** Backend and frontend methods must isolate core logic paths to allow regular execution of automated unit and validation scripts.

5.5 Business Rules

- Only verified accounts marked with Caregiver or Administrator roles can view patient metric dashboards or track geographic location coordinates.
- An automated alert notification will repeat every 3 minutes if it is not acknowledged or cleared on the primary caregiver console.
- New registration profiles must confirm their target email access addresses before the application layer unlocks active streaming features.
- Once an alert event status transitions to "Cleared" or "Resolved" by an administrator, its historical records are locked to prevent retroactive editing.

6. Other Requirements

6.1 Database Requirements

The system utilizes Post-gre SQL as its primary database system, supplemented with relational optimization and indexing layers for handling rapid time-series health entries. The storage environment tracks user accounts, configuration records, alert states, threshold parameters, and historical metric entries. Relational tables use strict third-normal-form layout rules to optimize storage efficiency and prevent performance bottlenecks. Automated data backup routines are scheduled to export data profiles to cloud storage buckets daily.

6.2 Internationalization Requirements

The system's text and message configurations default to English. System strings, dashboard readouts, and alert template engines use standard localization keys, making it straightforward to add regional language packs via simple locale configuration updates down the line.

6.3 Legal and Regulatory Requirements

ECMS must compile with essential data privacy acts under country-specific frameworks (e.g., India's IT Act 2000/2008 and Digital Personal Data Protection mandates) regarding the collection and management of personal tracking data. The platform must guarantee the confidentiality of sensitive patient profiles and log access histories securely.

6.4 Reuse Objectives

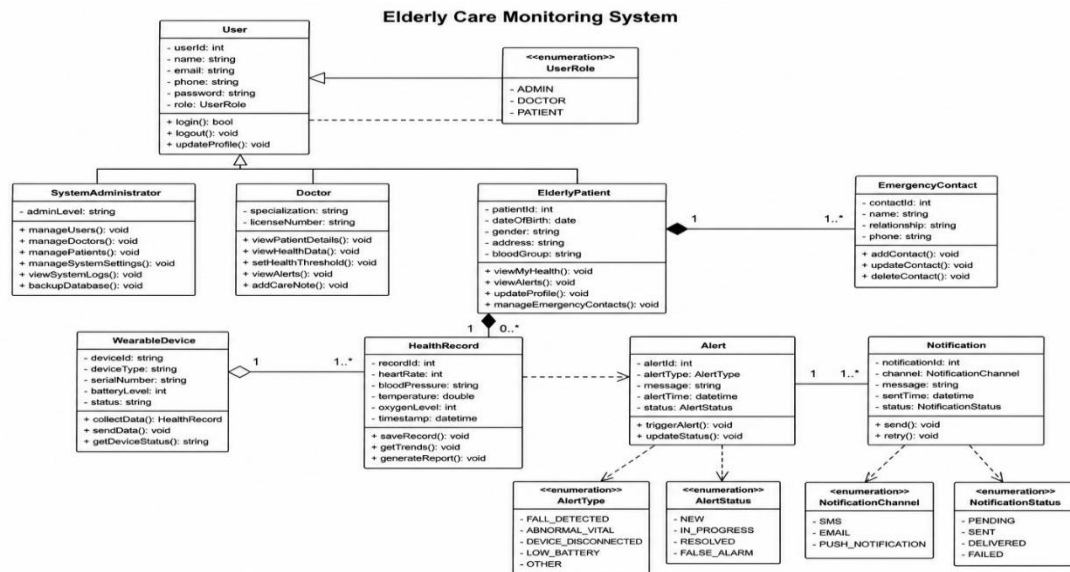
The application layer components—specifically the real-time Web Socket communication infrastructure, the JWT security middleware, and the multi-channel notification engine—must be designed using decoupled modules to facilitate easy integration into future tracking systems.

Appendix A: Glossary

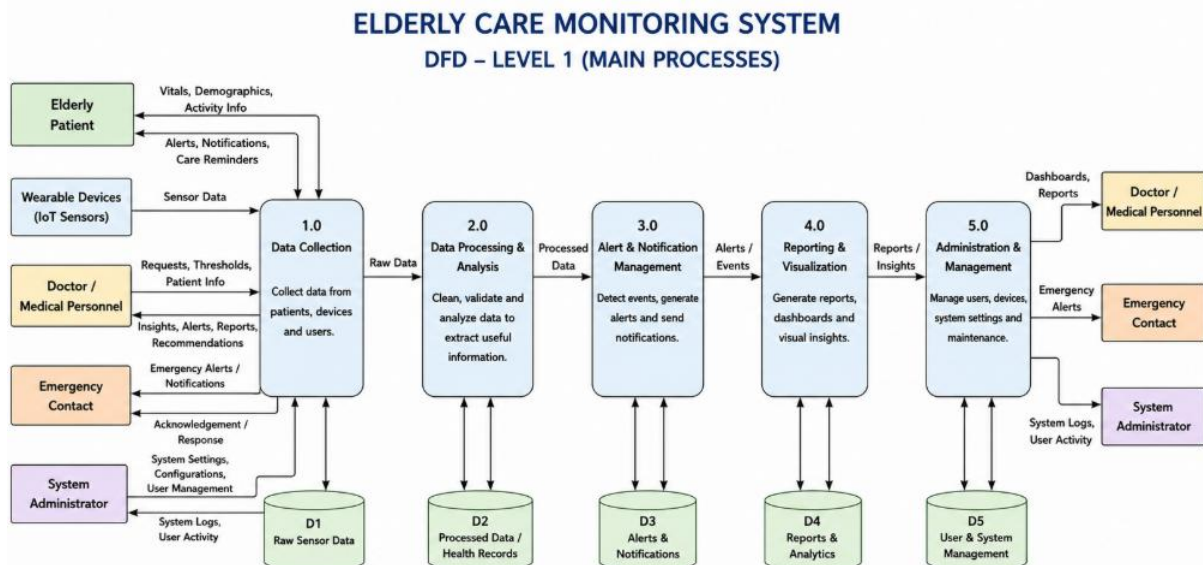
- **Caregiver:** An authorized system user with access rights to monitor assigned patient records and modify tracking thresholds.
- **Elderly Profile:** A core relational database model tracking biometric metrics and configuration history for a monitored individual.
- **Vital Parameter:** Ingested time-series records tracking body data points, including heart rate (\$HR\$), oxygen saturation (\$SpO_2\$), and temperature.
- **Geo-fence:** A virtual geometric boundary coordinate array stored in the database to restrict or validate target tracking movements.
- **Backend Ingestion Layer:** The asynchronous server-side codebase built on FastAPI to ingest payloads and execute logic.
- **Frontend Portal:** The browser and mobile interface framework compiled in React/Flutter that users interact with.
- **Time-ScaleDB:** A time-series database plugin for PostgreSQL used to manage high-frequency tracking histories efficiently.
- **JWT (JSON Web Token):** A secure, compact string format used to manage user authentication variables during active sessions.

Appendix B: Analysis Models

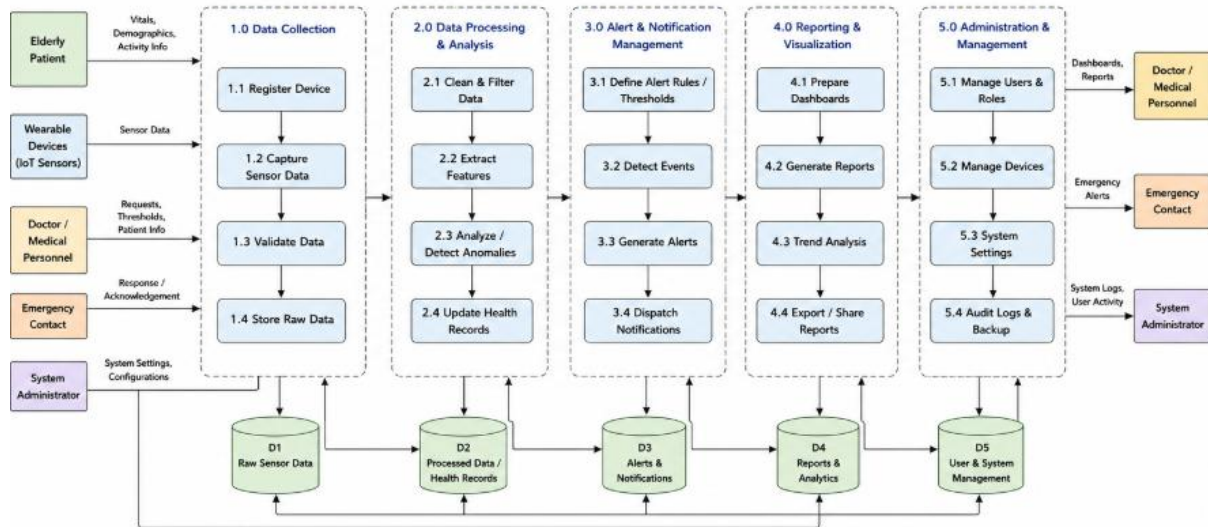
1. UML Class Diagram



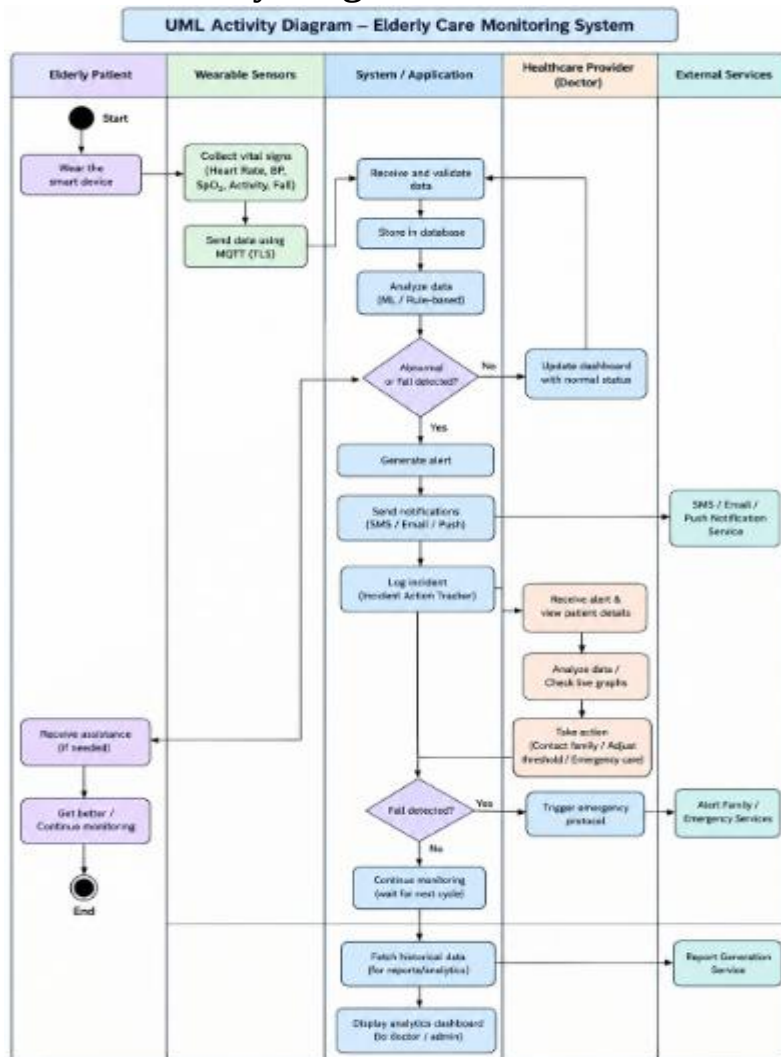
2. Data Flow Diagram (DFD) – Level 1 and Level 2



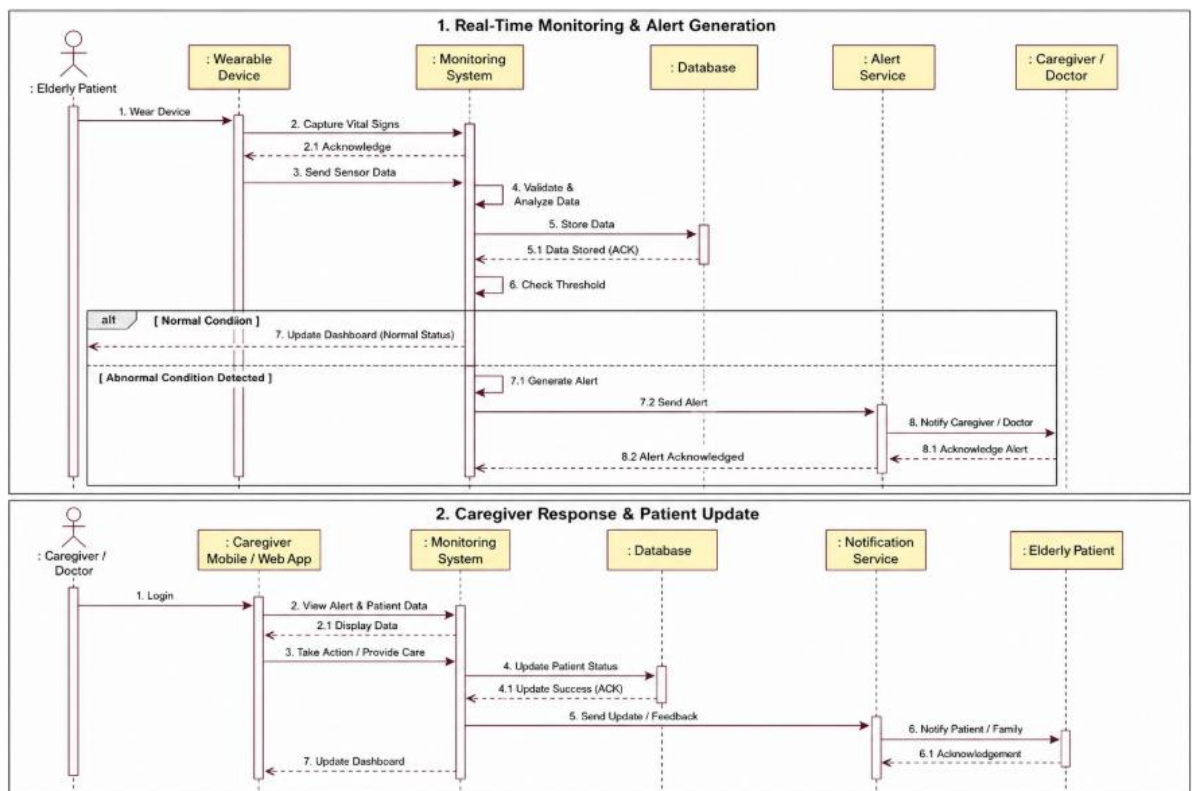
ELDERLY CARE MONITORING SYSTEM DFD – LEVEL 2 (DECOMPOSITION OF PROCESSES)



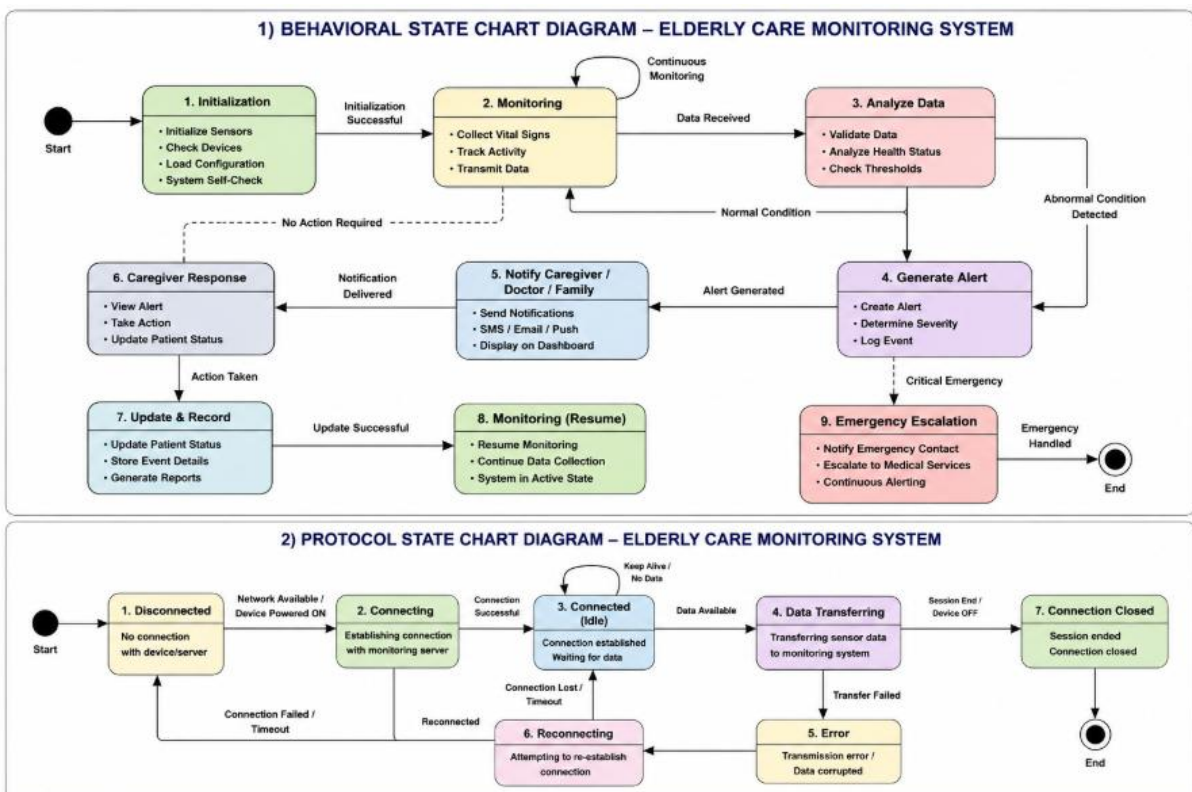
4.UML Activity Diagram



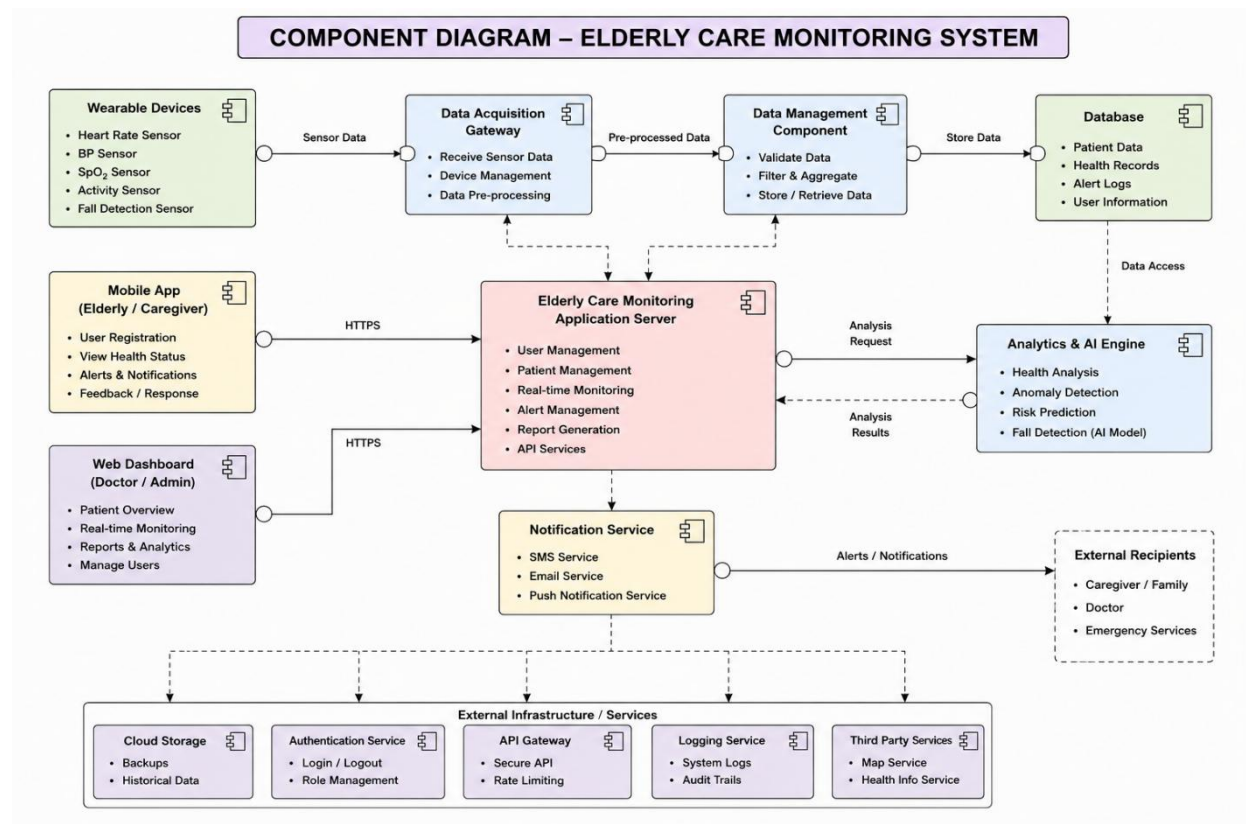
5. UML Sequence Diagram



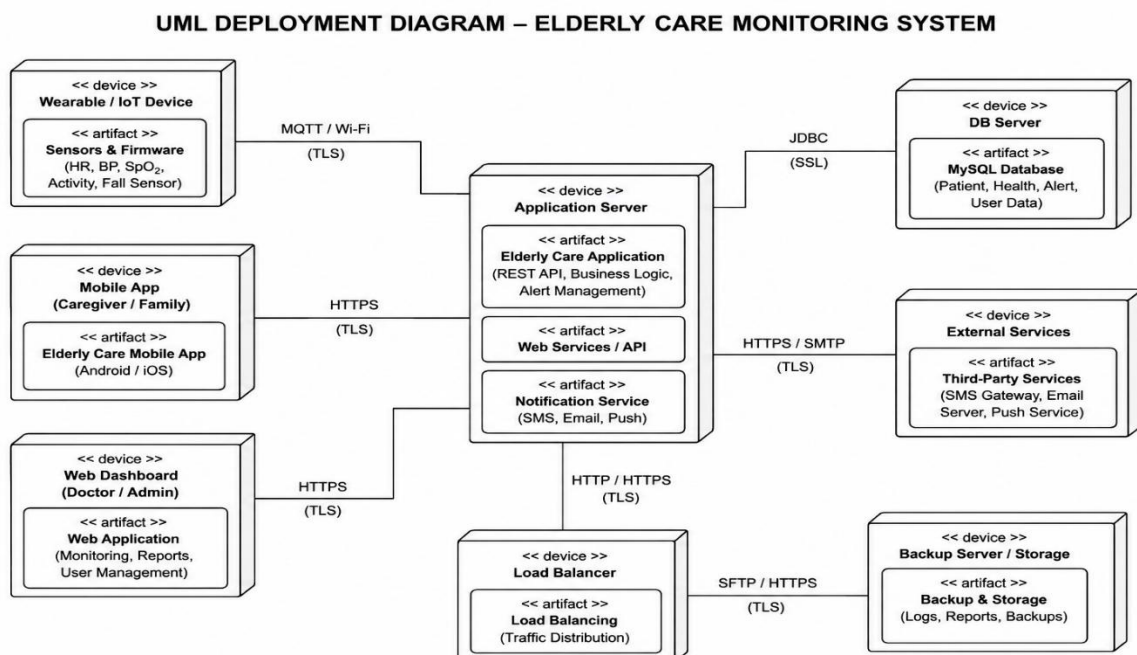
6. State Chart Diagram



7. UML Component



8. UML Deployment



Demonstration:

Code:

App.js

```
let currentRole = "caregiver";
let ws = null;
let vitalsChart = null;
let map = null;
let patientMarker = null;
let geofenceCircle = null;
let audioEnabled = true;
let audioCtx = null;
let autoStreamTimer = null;
let activeCountdownAlertId = null;
let patientConfig = {
  min_hr: 55,
  max_hr: 105,
  min_spo2: 92.0,
  safe_lat: 11.0168,
  safe_lng: 76.9558,
  safe_radius_meters: 250.0
};
document.addEventListener("DOMContentLoaded", async () => {
  initChart();
  initMap();
  await fetchPatientData();
  await loadMetricHistory();
  await loadIncidentAlerts();
  await loadDispatchLogs();
  connectWebSocket();
});

function playBeep(freq = 880, duration = 0.15, type = "sine") {
  if (!audioEnabled) return;
  try {
    if (!audioCtx) {
      audioCtx = new (window.AudioContext || window.webkitAudioContext)();
    }
    if (audioCtx.state === "suspended") {
      audioCtx.resume();
    }
    const osc = audioCtx.createOscillator();
    const gain = audioCtx.createGain();
    osc.type = type;
    osc.frequency.setValueAtTime(freq, audioCtx.currentTime);
    gain.gain.setValueAtTime(0.15, audioCtx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.001, audioCtx.currentTime + duration);
    osc.connect(gain);
    gain.connect(audioCtx.destination);
    osc.start();
    osc.stop(audioCtx.currentTime + duration);
  } catch (e) {
    console.warn("Audio Context init error", e);
  }
}

function toggleAudio() {
  audioEnabled = !audioEnabled;
  const btn = document.getElementById("soundToggleBtn");
  btn.textContent = audioEnabled ? "🔊 Audio Alerts: ON" : "🔇 Audio Alerts: OFF";
}

function switchRole(role) {
  currentRole = role;
  console.log("Switched view to role: ${role}");
  const titleRow = document.querySelector(".brand-subtitle");
  if (role === "doctor") {
    titleRow.textContent = "Medical Personnel & Clinical Diagnostics Portal";
  } else if (role === "admin") {
    titleRow.textContent = "System Administration & Infrastructure Operations";
  } else {
    titleRow.textContent = "Family Caregiver Live Monitoring & Safety Center";
  }
  loadIncidentAlerts();
}

let _wsReconnectDelay = 1000; // Start at 1s, exponential backoff up to 30s
let _wsHeartbeatTimer = null;
function connectWebSocket() {
  const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
  const wsUrl = `${protocol}//${window.location.host}/ws`;
  const pill = document.getElementById("connectionPill");
  const statusText = document.getElementById("connStatusText");
  if (_wsHeartbeatTimer) {
    clearInterval(_wsHeartbeatTimer);
    _wsHeartbeatTimer = null;
  }
  try {
    ws = new WebSocket(wsUrl);
  } catch (e) {
    console.error("WebSocket creation failed:", e);
    _scheduleReconnect(pill, statusText);
    return;
  }
  ws.onopen = () => {
    pill.className = "status-pill status-connected";
    statusText.textContent = "Live WS Connected";
    _wsReconnectDelay = 1000; // Reset backoff on successful connect
    console.log("WebSocket connected to ECMS backend");
    _wsHeartbeatTimer = setInterval(() => {
      if (ws.readyState === WebSocket.OPEN) {
        ws.send("ping");
      }
    }, 20000);
  };
  ws.onmessage = (event) => {
    if (event.data === "pong") return;
    try {
      const data = JSON.parse(event.data);
      handleIncomingSocketMessage(data);
    } catch (err) {
      console.error("Error parsing WS frame:", err);
    }
  };
  ws.onclose = (event) => {
    pill.className = "status-pill status-connecting";
    console.warn("WebSocket closed (code: ${event.code}). Reconnecting in ${_wsReconnectDelay / 1000}s...");
    _scheduleReconnect(pill, statusText);
  };
  ws.onerror = (err) => {
    console.error("WebSocket error:", err);
    ws.close();
  };
}

function _scheduleReconnect(pill, statusText) {
  if (_wsHeartbeatTimer) {
    clearInterval(_wsHeartbeatTimer);
    _wsHeartbeatTimer = null;
  }
  const delay = _wsReconnectDelay;
  statusText.textContent = `Reconnecting in ${Math.round(delay / 1000)}s...`;
  setTimeout(connectWebSocket, delay);
  _wsReconnectDelay = Math.min(_wsReconnectDelay * 2 + Math.random() * 500, 30000);
}

function handleIncomingSocketMessage(data) {
  switch (data.type) {
    case "NEW_METRIC":
      updateMetricDisplay(data.metric, data.anomalies, data.is_fall);
      appendChartData(data.metric);
      updateMapPosition(data.metric.latitude, data.metric.longitude);
      if (data.anomalies.length > 0) {
        playBeep(700, 0.25, "triangle");
        loadIncidentAlerts();
      }
      break;
    case "COUNTDOWN_TICK":
      handleCountdownTick(data.alert_id, data.remaining_seconds);
      break;
    case "ALERT_CANCELLED":
      handleAlertCancelled(data.alert_id);
      break;
    case "EMERGENCY_CONFIRMED":
      handleEmergencyConfirmed(data.alert_id, data.message);
      break;
    case "ALERT_RESOLVED":
      loadIncidentAlerts();
      break;
    case "RESET_ALL":
      resetUiDisplay();
      break;
  }
}

function updateMetricDisplay(metric, anomalies = [], is_fall = false) {
  document.getElementById("valHeartRate").textContent = metric.heart_rate;
  document.getElementById("valSpO2").textContent = metric.spo2.toFixed(1);
  document.getElementById("valTemp").textContent = metric.temperature.toFixed(1);
  document.getElementById("valMotion").textContent = metric.motion_vector.toFixed(2);
  document.getElementById("valSteps").textContent = metric.steps.toLocaleString();
  document.getElementById("lastSyncTime").textContent = metric.timestamp;
  const hrStatus = document.getElementById("hrStatus");
  if (metric.heart_rate > patientConfig.max_hr) {
    hrStatus.textContent = "Tachycardia High";
    hrStatus.className = "badge badge-emergency";
  } else if (metric.heart_rate < patientConfig.min_hr) {
    hrStatus.textContent = "Bradycardia Low";
    hrStatus.className = "badge badge-warning";
  } else {
    hrStatus.textContent = "Normal";
    hrStatus.className = "badge badge-success";
  }
  const spo2Status = document.getElementById("spo2Status");
  if (metric.spo2 < patientConfig.min_spo2) {
    spo2Status.textContent = "Hypoxemia Low";
    spo2Status.className = "badge badge-emergency";
  } else {
    spo2Status.textContent = "Optimal";
    spo2Status.className = "badge badge-success";
  }
  const motionStatus = document.getElementById("motionStatus");
  if (metric.motion_vector >= 3.2 || is_fall) {
    motionStatus.textContent = "HIGH IMPACT FALL";
    motionStatus.className = "badge badge-emergency";
  } else {
    motionStatus.textContent = "Steady";
    motionStatus.className = "badge badge-success";
  }
  const overallBadge = document.getElementById("patientStatusBadge");
  if (is_fall || (anomalies.length > 0)) {
    overallBadge.textContent = "Status: Warning Anomaly";
    overallBadge.className = "badge badge-warning";
  } else {
    overallBadge.textContent = "Status: Normal";
    overallBadge.className = "badge badge-normal";
  }
}
```

```

function handleCountdownTick(alertId, remaining) {
  activeCountdownAlertId = alertId;
  const modal = document.getElementById("fallCountdownModal");
  const numDisplay = document.getElementById("countdownNumber");
  const strokeCircle = document.getElementById("countdownStroke");
  modal.classList.remove("hidden");
  numDisplay.textContent = remaining;
  const circumference = 283;
  const offset = circumference - (remaining / 15) * circumference;
  strokeCircle.style.strokeDashoffset = offset;
  playBeep(920, 0.12, "square");
  const badge = document.getElementById("patientStatusBadge");
  badge.textContent = `Fall Verification: ${remaining}s`;
  badge.className = "badge badge-emergency";
}
async function cancelActiveAlert() {
  if (!activeCountdownAlertId) return;
  try {
    const res = await fetch(`/api/alerts/${activeCountdownAlertId}/cancel`, { method: "POST" });
    const data = await res.json();
    console.log("Alert cancel response:", data);
    handleAlertCancelled(activeCountdownAlertId);
  } catch (err) {
    console.error("Failed to cancel alert", err);
  }
}
function handleAlertCancelled(alertId) {
  activeCountdownAlertId = null;
  const modal = document.getElementById("fallCountdownModal");
  modal.classList.add("hidden");
  const badge = document.getElementById("patientStatusBadge");
  badge.textContent = "Status: Normal (False Alarm Cancelled)";
  badge.className = "badge badge-normal";

  playBeep(440, 0.3, "sine"); // reassuring low tone
  loadIncidentAlerts();
}
function handleEmergencyConfirmed(alertId, message) {
  activeCountdownAlertId = null;
  const modal = document.getElementById("fallCountdownModal");
  modal.classList.add("hidden");
  const badge = document.getElementById("patientStatusBadge");
  badge.textContent = "STATUS: CRITICAL EMERGENCY DISPATCHED";
  badge.className = "badge badge-emergency";
  playBeep(1200, 0.6, "sawtooth");
  loadIncidentAlerts();
  loadDispatchLogs();
}
function initChart() {
  const ctx = document.getElementById("vitalsChart").getContext("2d");
  vitalsChart = new Chart(ctx, {
    type: "line",
    data: {
      labels: [],
      datasets: [
        {
          label: "Heart Rate (bpm)",
          data: [],
          borderColor: "#f43f5e",
          backgroundColor: "rgba(244, 63, 94, 0.08)",
          borderWidth: 2.5,
          tension: 0.35,
          fill: true,
          pointRadius: 3,
          pointHoverRadius: 6,
          yAxisID: "y"
        },
        {
          label: "SpO2 (%)",
          data: [],
          borderColor: "#06b6d4",
          backgroundColor: "transparent",
          borderWidth: 2,
          borderDash: [4, 4],
          tension: 0.3,
          pointRadius: 2,
          yAxisID: "y1"
        }
      ],
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      interaction: {
        mode: "index",
        intersect: false
      },
      plugins: {
        legend: {
          labels: { color: "#94a3b8", font: { family: "Inter", size: 11 } }
        }
      },
      scales: {
        x: {
          grid: { color: "rgba(255, 255, 255, 0.05)" },
          ticks: { color: "#64748b", font: { size: 10 }, maxTicksLimit: 6 }
        },
        y: {
          type: "linear",
          position: "left",
          min: 40,
          max: 160,
          grid: { color: "rgba(255, 255, 255, 0.05)" },
          ticks: { color: "#f43f5e", font: { size: 10 } },
          title: { display: true, text: "BPM", color: "#f43f5e" }
        },
        y1: {
          type: "linear",
          position: "right",
          min: 75,
          max: 100,
          grid: { drawOnChartArea: false },
          ticks: { color: "#06b6d4", font: { size: 10 } },
          title: { display: true, text: "SpO2 %", color: "#06b6d4" }
        }
      }
    }
  });
}
function appendChartData(metric) {
  if (!vitalsChart) return;
  const timeLabel = metric.timestamp.split(" ")[1] || metric.timestamp;
  vitalsChart.data.labels.push(timeLabel);
  vitalsChart.data.datasets[0].data.push(metric.heart_rate);
  vitalsChart.data.datasets[1].data.push(metric.spo2);
  if (vitalsChart.data.labels.length > 20) {
    vitalsChart.data.labels.shift();
    vitalsChart.data.datasets[0].data.shift();
    vitalsChart.data.datasets[1].data.shift();
  }
  vitalsChart.update("none");
}
async function loadMetricHistory() {
  try {
    const res = await fetch("/api/history?limit=15");
    const history = await res.json();
    if (Array.isArray(history)) {
      history.forEach(m => {
        const timeLabel = m.timestamp.split(" ")[1] || m.timestamp;
        vitalsChart.data.labels.push(timeLabel);
        vitalsChart.data.datasets[0].data.push(m.heart_rate);
        vitalsChart.data.datasets[1].data.push(m.spo2);
      });
      vitalsChart.update();
    }
  } catch (e) {
    console.error("Failed to load metric history:", e);
  }
}
function initMap() {
  const lat = patientConfig.safe_lat;
  const lng = patientConfig.safe_lng;
  map = L.map("mapContainer", { zoomControl: false }).setView([lat, lng], 16);

  L.tileLayer("https://s.tile.openstreetmap.org/{z}/{x}/{y}.png", {
    attribution: "&copy; <a href='https://www.openstreetmap.org/copyright'>OpenStreetMap</a> contributors'");
  }).addTo(map);
  L.control.zoom({ position: "bottomright" }).addTo(map);
  geofenceCircle = L.circle([lat, lng], {
    color: "#10b981",
    fillColor: "#10b981",
    fillOpacity: 0.18,
    radius: patientConfig.safe_radius_meters
  }).addTo(map);
  const patientIcon = L.divIcon({
    className: "custom-patient-pin",
    html: '<div style="background:#2563eb; width:22px; height:22px; border-radius:50%; border:3px solid #ffffff; box-shadow:0 0 12px #2563eb;"></div>',
    iconSize: [22, 22],
    iconAnchor: [11, 11]
  });
  patientMarker = L.marker([lat, lng], { icon: patientIcon }).addTo(map);
  patientMarker.bindPopup("<b>Robert Jenkins</b><br>Safe Zone Tracker").openPopup();
}
function updateMapPosition(lat, lng) {
  if (!map || !patientMarker) return;
  patientMarker.setLatLng([lat, lng]);
  document.getElementById("mapLat").textContent = `${lat.toFixed(4)}° N`;
  document.getElementById("mapLng").textContent = `${lng.toFixed(4)}° E`;
  const safeLatLng = L.latLng(patientConfig.safe_lat, patientConfig.safe_lng);
  const currentLatLng = L.latLng(lat, lng);
  const dist = safeLatLng.distanceTo(currentLatLng);
  const pill = document.getElementById("geofenceStatusPill");
  const txt = document.getElementById("geofenceText");
  if (dist > patientConfig.safe_radius_meters) {
    pill.className = "status-pill status-danger";
    txt.textContent = `BREACH: ${Math.round(dist)}m away`;
    if (geofenceCircle) {
      geofenceCircle.setStyle({ color: "#ef4444", fillColor: "#ef4444" });
    }
  } else {
    pill.className = "status-pill status-safe";
    txt.textContent = "Safe in Zone";
    if (geofenceCircle) {
      geofenceCircle.setStyle({ color: "#10b981", fillColor: "#10b981" });
    }
  }
}
async function loadIncidentAlerts() {
  try {
    const res = await fetch("/api/alerts");
    const alerts = await res.json();
    const tbody = document.getElementById("incidentTableBody");
    tbody.innerHTML = "";
    if (!alerts || alerts.length === 0) {
      tbody.innerHTML = "<tr><td colspan='7' class='text-center text-muted'>No safety incidents recorded.</td></tr>";
      return;
    }
    alerts.forEach(a => {
      const tr = document.createElement("tr");
      let statusBadge = "badge-normal";
      if (a.status === "Emergency") statusBadge = "badge-emergency";
      else if (a.status === "Verifying_Countdown") statusBadge = "badge-warning";
      else if (a.status === "Cancelled") statusBadge = "badge-age";
      let actionBtn = "";
      if (a.status === "Emergency") {
        actionBtn = "<button class='btn btn-sm btn-primary'>";
        onclick = "openResolveModal(${a.id})">Resolve & Sign Off</button>";
      } else if (a.status === "Resolved") {
        actionBtn = "<span class='text-muted' title='Historical records locked'> Locked</span>";
      } else {
        actionBtn = "<span class='text-muted'></span>";
      }
      tr.innerHTML = `
        <td>#${a.id}</td>
        <td><strong>${a.alert_type}</strong></td>
      `;
    });
  }
}

```

```

<td><span class="badge ${a.severity} === 'Emergency' ? 'badge-emergency' : 'badge-
warning'">${a.severity}</span></td>
<td>${a.message}</td>
<td><span class="badge ${statusBadge}">${a.status}</span></td>
<td>${a.doctor_notes} || <em class="text-muted">Pending review</em></td>
<td>${actionBtn}</td>
</tr>
tbody.appendChild(tr);
});
} catch (e) {
  console.error("Failed to load incidents:", e);
}
}
function openResolveModal(alertId) {
  document.getElementById("resolveAlertId").value = alertId;
  document.getElementById("doctorNotesInput").value = "";
  document.getElementById("resolveModal").classList.remove("hidden");
}
function closeResolveModal() {
  document.getElementById("resolveModal").classList.add("hidden");
}
async function submitIncidentResolution(event) {
  event.preventDefault();
  const alertId = document.getElementById("resolveAlertId").value;
  const notes = document.getElementById("doctorNotesInput").value;
  try {
    const res = await fetch(`/api/alerts/${alertId}/resolve`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ doctor_notes: notes })
    });
    if (res.ok) {
      closeResolveModal();
      loadIncidentAlerts();
    }
  } catch (e) {
    console.error("Failed to resolve alert:", e);
  }
}
async function loadDispatchLogs() {
  try {
    const res = await fetch("/api/dispatch-logs");
    const logs = await res.json();
    const container = document.getElementById("dispatchLogList");
    if (!logs || logs.length === 0) {
      container.innerHTML = `<div class="empty-state">No emergency dispatches issued yet.
System running in safe baseline.</div>`;
      return;
    }
    container.innerHTML = "";
    logs.forEach(log => {
      const item = document.createElement("div");
      item.className = "dispatch-item";
      item.innerHTML = `
<div class="dispatch-meta">
<span class="dispatch-channel">${log.channel}</span> & rarr; ${log.recipient}</span>
<span>${log.dispatched_at}</span>
</div>
<div class="dispatch-payload">${log.payload}</div>
`;
      container.appendChild(item);
    });
  } catch (e) {
    console.error("Failed to load dispatch logs:", e);
  }
}
async function fetchPatientData() {
  try {
    const res = await fetch("/api/patient");
    const data = await res.json();
    if (data.profile) {
      document.getElementById("patientName").textContent = data.profile.full_name;
      document.getElementById("patientAge").textContent = `Age: ${data.profile.age}`;
      document.getElementById("patientHistory").textContent = `Conditions:
${data.profile.medical_history}`;
      document.getElementById("patientEmergencyContact").textContent = `
${data.profile.emergency_contact} (Primary Caregiver)`;
    }
    if (data.thresholds) {
      patientConfig = data.thresholds;
      document.getElementById("hrThresholdText").textContent = `Safe Range:
${patientConfig.min_hr} - ${patientConfig.max_hr} bpm`;
      document.getElementById("spo2ThresholdText").textContent = `Safe: ≥
${patientConfig.min_spo2}%`;
      document.getElementById("mapRadius").textContent = `
${patientConfig.safe_radius_meters} meters`;
    }
    if (data.latest_metric) {
      updateMetricDisplay(data.latest_metric);
      updateMapPosition(data.latest_metric.latitude, data.latest_metric.longitude);
    }
  } catch (e) {
    console.error("Failed to fetch patient data:", e);
  }
}
function openThresholdModal() {
  document.getElementById("inputMinHr").value = patientConfig.min_hr;
  document.getElementById("inputMaxHr").value = patientConfig.max_hr;
  document.getElementById("inputMinSpO2").value = patientConfig.min_spo2;
  document.getElementById("inputRadius").value = patientConfig.safe_radius_meters;
  document.getElementById("inputSafeLat").value = patientConfig.safe_lat;
  document.getElementById("inputSafeLng").value = patientConfig.safe_lng;
  document.getElementById("thresholdModal").classList.remove("hidden");
}
function closeThresholdModal() {
  document.getElementById("thresholdModal").classList.add("hidden");
}
async function saveThresholds(event) {
  event.preventDefault();
  const updated = {
    min_hr: parseInt(document.getElementById("inputMinHr").value),
    max_hr: parseInt(document.getElementById("inputMaxHr").value),
    min_spo2: parseFloat(document.getElementById("inputMinSpO2").value),
    min_temp: 36.0,

```

```

    max_temp: 37.8,
    safe_lat: parseFloat(document.getElementById("inputSafeLat").value),
    safe_lng: parseFloat(document.getElementById("inputSafeLng").value),
    safe_radius_meters: parseFloat(document.getElementById("inputRadius").value)
  };
  try {
    const res = await fetch("/api/thresholds", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(updated)
    });
    if (res.ok) {
      patientConfig = updated;
      document.getElementById("hrThresholdText").textContent = `Safe Range:
${patientConfig.min_hr} - ${patientConfig.max_hr} bpm`;
      document.getElementById("spo2ThresholdText").textContent = `Safe: ≥
${patientConfig.min_spo2}%`;
      document.getElementById("mapRadius").textContent = `
${patientConfig.safe_radius_meters} meters`;
      if (geofenceCircle) {
        geofenceCircle.setRadius(patientConfig.safe_radius_meters);
        geofenceCircle.setLatLng([patientConfig.safe_lat, patientConfig.safe_lng]);
      }
      closeThresholdModal();
    }
  } catch (e) {
    console.error("Failed to save thresholds:", e);
  }
}
function toggleAutoStream() {
  const btn = document.getElementById("autoStreamBtn");
  if (autoStreamTimer) {
    clearInterval(autoStreamTimer);
    autoStreamTimer = null;
    btn.textContent = `▶ Start Auto Stream`;
    btn.classList.remove("btn-stream-active");
  } else {
    btn.textContent = `□ Stop Auto Stream`;
    btn.classList.add("btn-stream-active");
    autoStreamTimer = setInterval(sendRandomHealthyMetric, 2500);
    sendRandomHealthyMetric();
  }
}
async function sendMetricPayload(payload) {
  try {
    const res = await fetch("/api/metrics", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload)
    });
    return await res.json();
  } catch (e) {
    console.error("Failed to push metric payload:", e);
  }
}
function sendRandomHealthyMetric() {
  const hr = Math.floor(70 + Math.random() * 10);
  const spo2 = +(97 + Math.random() * 2).toFixed(1);
  const temp = +(36.4 + Math.random() * 0.4).toFixed(1);
  const steps = parseInt(document.getElementById("valSteps").textContent.replace(",", "")) +
Math.floor(Math.random() * 4);
  const lat = patientConfig.safe_lat + (Math.random() - 0.5) * 0.0003;
  const lng = patientConfig.safe_lng + (Math.random() - 0.5) * 0.0003;
  sendMetricPayload({
    patient_id: 1,
    heart_rate: hr,
    spo2: spo2,
    temperature: temp,
    steps: steps,
    latitude: lat,
    longitude: lng,
    motion_vector: 0.98 + (Math.random() * 0.1)
  });
}
function triggerFallSimulation() {
  fetch("/api/simulate/fall", { method: "POST" });
}
function simulateTachycardia() {
  sendMetricPayload({
    patient_id: 1,
    heart_rate: 135,
    spo2: 96.5,
    temperature: 36.8,
    steps: 2750,
    latitude: patientConfig.safe_lat,
    longitude: patientConfig.safe_lng,
    motion_vector: 1.15
  });
}
function simulateLowSpO2() {
  sendMetricPayload({
    patient_id: 1,
    heart_rate: 88,
    spo2: 84.0, // Hypoxemia!
    temperature: 36.5,
    steps: 2750,
    latitude: patientConfig.safe_lat,
    longitude: patientConfig.safe_lng,
    motion_vector: 0.92
  });
}
function simulateGeofenceBreach() {
  const breachLat = patientConfig.safe_lat + 0.0050;
  const breachLng = patientConfig.safe_lng + 0.0050;
  sendMetricPayload({
    patient_id: 1,
    heart_rate: 82,
    spo2: 97.0,
    temperature: 36.6,
    steps: 3100,
    latitude: breachLat,
    longitude: breachLng,
    motion_vector: 1.05
  });
}

```

```
async function resetToNormal() {
  if (autoStreamTimer) {
    clearInterval(autoStreamTimer);
    autoStreamTimer = null;
    document.getElementById("autoStreamBtn").textContent = "► Start Auto Stream";
  }
  await fetch("/api/simulate/reset", { method: "POST" });
  await fetchPatientData();
  await loadIncidentAlerts();
  await loadDispatchLogs();
}
function resetUiDisplay() {
  document.getElementById("patientStatusBadge").textContent = "Status: Normal";
  document.getElementById("patientStatusBadge").className = "badge badge-normal";
  document.getElementById("fallCountdownModal").classList.add("hidden");
  if (geofenceCircle) {
    geofenceCircle.setStyle({ color: "#10b981", fillColor: "#10b981" });
  }
}
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Elderly Care Monitoring System (ECMS) - Live Dashboard</title>
  <!-- Google Fonts: Outfit & Inter for ultra-clean modern typography -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@400;500;600;700;800&family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
  <!-- Leaflet CSS for Map & Geo-fencing -->
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" integrity="sha256-p4NxAoJBhIIN+hmNHrzRCf9tD/miZyoHS5obTRR9BMY=" crossorigin="" />
  <!-- Chart.js for Health Trends -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <!-- Leaflet JS -->
  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js" integrity="sha256-20nQCChB9co0qIjJZRGuk2/Z9VM+kNiyxNV1vTlZBo=" crossorigin=""></script>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body class="theme-dark">
  <!-- Top Persistent Navigation Bar -->
  <header class="navbar">
    <div class="nav-brand">
      <div class="brand-logo">
        <span class="heart-pulse-icon"></span>
      </div>
      <div>
        <h1 class="brand-title">ECMS <span>CarePortal</span></h1>
        <p class="brand-subtitle">Elderly Care & Emergency Fall Monitoring System</p>
      </div>
    </div>
    <div class="nav-controls">
      <!-- Role-Based View Switcher -->
      <div class="role-selector-wrap">
        <label for="roleSelector">Current View:</label>
        <select id="roleSelector" onchange="switchRole(this.value)">
          <option value="doctor"><input checked="" type="checkbox"/> Dr. Arvind Sharma (Doctor)</option>
          <option value="caregiver" selected>Priya K (Family Caregiver)</option>
          <option value="admin"><input type="checkbox"/> Kathiravan A (Administrator)</option>
        </select>
      </div>
      <!-- WebSocket Connection Status Indicator -->
      <div id="connectionPill" class="status-pill status-connecting">
        <span class="dot"></span>
        <span id="connStatusText">Connecting...</span>
      </div>
      <!-- Sound Alert Toggle -->
      <button id="soundToggleBtn" class="btn btn-outline" onclick="toggleAudio()"
title="Toggle Emergency Siren Sound">
        Audio Alerts: ON
      </button>
    </div>
  </header>
  <div class="app-layout">
    <!-- Main Dashboard Viewport -->
    <main class="dashboard-content">
      <!-- Patient Banner Card -->
      <section class="patient-card">
        <div class="patient-avatar-box">
          <div class="patient-avatar"></div>
          <div>
            <div class="patient-title-row">
              <h2 id="patientName">Robert Jenkins</h2>
              <span class="badge badge-age" id="patientAge">Age: 78</span>
              <span id="patientStatusBadge" class="badge badge-normal">Status:
Normal</span>
            </div>
            <p class="patient-history" id="patientHistory">Conditions: Hypertension, Mild
Cardiac Arrhythmia, Osteoporosis</p>
          </div>
          <div>
            <div class="patient-quick-stats">
              <div class="stat-pill">
                <span class="label">Emergency Contact:</span>
                <strong id="patientEmergencyContact">+91-9876500003 (Priya K)</strong>
              </div>
              <div class="stat-pill">
                <span class="label">Last Ingested Sync:</span>
                <strong id="lastSyncTime">Waiting for data...</strong>
              </div>
              <button class="btn btn-primary btn-sm" onclick="openThresholdModal()">Threshold Settings</button>
            </div>
          </div>
        </div>
      </section>
      <!-- Real-Time Metric Telemetry Cards Grid -->
      <section class="metrics-grid">
```

```
<!-- Heart Rate Widget -->
<div class="metric-card" id="cardHeartRate">
  <div class="metric-header">
    <span class="metric-title">Heart Rate (HR)</span>
    <span class="card-icon heart-icon"></span>
  </div>
  <div class="metric-body">
    <span class="metric-value" id="valHeartRate">72</span>
    <span class="metric-unit">bpm</span>
  </div>
  <div class="metric-footer">
    <span class="threshold-tag" id="hrThresholdText">Safe Range: 55 - 105
bpm</span>
  </div>
  <div>
    <span class="badge badge-success" id="hrStatus">Normal</span>
  </div>
</div>
<!-- SpO2 Oxygen Widget -->
<div class="metric-card" id="cardSpO2">
  <div class="metric-header">
    <span class="metric-title">Blood Oxygen (SpO2)</span>
    <span class="card-icon"></span>
  </div>
  <div class="metric-body">
    <span class="metric-value" id="valSpO2">98.0</span>
    <span class="metric-unit">%</span>
  </div>
  <div class="metric-footer">
    <span class="threshold-tag" id="spo2ThresholdText">Safe: &ge; 92.0%</span>
    <span class="badge badge-success" id="spo2Status">Optimal</span>
  </div>
</div>
<!-- Body Temperature Widget -->
<div class="metric-card" id="cardTemp">
  <div class="metric-header">
    <span class="metric-title">Body Temperature</span>
    <span class="card-icon"></span>
  </div>
  <div class="metric-body">
    <span class="metric-value" id="valTemp">36.6</span>
    <span class="metric-unit">°C</span>
  </div>
  <div class="metric-footer">
    <span class="threshold-tag">Range: 36.0 - 37.8 °C</span>
    <span class="badge badge-success" id="tempStatus">Normal</span>
  </div>
</div>
<!-- Daily Steps & Motion Vector Widget -->
<div class="metric-card" id="cardMotion">
  <div class="metric-header">
    <span class="metric-title">Activity & Deceleration</span>
    <span class="card-icon"></span>
  </div>
  <div class="metric-body">
    <span class="metric-value" id="valMotion">0.98</span>
    <span class="metric-unit">g force</span>
  </div>
  <div class="metric-footer">
    <span class="threshold-tag">Steps: <strong id="valSteps">2,600</strong></span>
    <span class="badge badge-success" id="motionStatus">Steady</span>
  </div>
</div>
</section>
<!-- Middle Split: Real-Time Charts & Geo-Fence Map -->
<section class="visuals-split">
  <!-- Real-Time Trend Charts -->
  <div class="panel-card chart-panel">
    <div class="panel-header">
      <div>
        <h3 class="panel-title">Real-Time Vitals Trend</h3>
        <p class="panel-subtitle">Streaming time-series telemetry (Heart Rate &
SpO2)</p>
      </div>
      <div>
        <span class="live-dot-tag"><span class="pulse-dot"></span> LIVE
STREAM</span>
      </div>
      <div class="chart-container">
        <canvas id="vitalsChart"></canvas>
      </div>
    </div>
  </div>
  <!-- Live GPS & Geo-Fencing Map -->
  <div class="panel-card map-panel">
    <div class="panel-header">
      <div>
        <h3 class="panel-title">Live GPS & Geo-Fence Tracker</h3>
        <p class="panel-subtitle">Monitors senior within authorized safe radius</p>
      </div>
      <div id="geofenceStatusPill" class="status-pill status-safe">
        <span class="dot"></span>
        <span id="geofenceText">Safe in Zone</span>
      </div>
      <div>
        <div id="mapContainer" class="map-container"></div>
        <div class="map-info-bar">
          <span>Lat: <strong id="mapLat">11.0168° N</strong>, Lng: <strong
id="mapLng">76.9558° E</strong></span>
          <span>Safe Perimeter: <strong id="mapRadius">250 meters</strong></span>
        </div>
      </div>
    </div>
  </div>
</section>
<!-- Bottom Split: Incident Action Tracker & Emergency Notification Dispatch Log -->
<section class="tracker-split">
  <!-- Incident Action Tracker -->
  <div class="panel-card incident-panel">
    <div class="panel-header">
      <div>
        <h3 class="panel-title">Incident Action Tracker</h3>
        <p class="panel-subtitle">Clinical resolution logs, alarms, and doctor sign-
offs</p>
      </div>
      <div>
        <button class="btn btn-outline btn-sm" onclick="loadIncidentAlerts()">Refresh
Alerts</button>
      </div>
    </div>
  </div>
```

records...</td>

```
max="99">
```

onclick="cl

submitted, this incident will transition to **Resolved** and become permanently locked.

```
:root {
```

}

```

}
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: rgba(18, 24, 38, 0.85);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid var(--border-subtle);
  position: sticky;
  top: 0;
  z-index: 1000;
}
.nav-brand {
  display: flex;
  align-items: center;
  gap: 1rem;
}
.brand-logo {
  width: 44px;
  height: 44px;
  border-radius: var(--radius-md);
  background: linear-gradient(135deg, #f43f5e, #be123c);
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.4);
}
.heart-pulse-icon {
  font-size: 1.5rem;
  animation: heartPulse 1.2s infinite ease-in-out;
}
@keyframes heartPulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.18); }
}
.brand-title {
  font-family: var(--font-heading);
  font-size: 1.35rem;
  font-weight: 700;
  letter-spacing: -0.02em;
  color: #fff;
}
.brand-title span {
  color: #60a5fa;
  font-weight: 400;
}
.brand-subtitle {
  font-size: 0.78rem;
  color: var(--text-secondary);
}
.nav-controls {
  display: flex;
  align-items: center;
  gap: 1.25rem;
}
.role-selector-wrap {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  background: var(--bg-main);
  padding: 0.35rem 0.75rem;
  border-radius: var(--radius-sm);
  border: 1px solid var(--border-subtle);
}
.role-selector-wrap label {
  font-size: 0.8rem;
  color: var(--text-secondary);
}
.role-selector-wrap select {
  background: transparent;
  border: none;
  color: #38bdf8;
  font-size: 0.85rem;
  font-weight: 600;
  cursor: pointer;
  outline: none;
}
.role-selector-wrap select option {
  background: var(--bg-card);
  color: var(--text-primary);
}
.status-pill {
  display: inline-flex;
  align-items: center;
  gap: 0.45rem;
  padding: 0.35rem 0.85rem;
  border-radius: 9999px;
  font-size: 0.8rem;
  font-weight: 600;
}
.status-connecting {
  background: rgba(245, 158, 11, 0.15);
  color: #fbbf24;
  border: 1px solid rgba(245, 158, 11, 0.3);
}
.status-connected {
  background: rgba(16, 185, 129, 0.15);
  color: #34d399;
  border: 1px solid rgba(16, 185, 129, 0.3);
}
.status-safe {
  background: rgba(16, 185, 129, 0.15);
  color: #34d399;
  border: 1px solid rgba(16, 185, 129, 0.3);
}
.status-danger {
  background: rgba(239, 68, 68, 0.2);
  color: #f87171;
  border: 1px solid rgba(239, 68, 68, 0.4);
  animation: dangerBlink 1s infinite alternate;
}
}
@keyframes dangerBlink {
  0% { opacity: 0.75; }
  100% { opacity: 1; box-shadow: 0 0 12px rgba(239, 68, 68, 0.5); }
}
.dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: currentColor;
}
.app-layout {
  flex: 1;
  display: flex;
  flex-direction: column;
  padding-bottom: 90px; /* Space for sticky simulator bar */
}
.dashboard-content {
  max-width: 1440px;
  margin: 0 auto;
  width: 100%;
  padding: 1.75rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.75rem;
}
.patient-card {
  background: var(--bg-card);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-lg);
  padding: 1.25rem 1.75rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: var(--shadow-card);
  flex-wrap: wrap;
  gap: 1rem;
}
.patient-avatar-box {
  display: flex;
  align-items: center;
  gap: 1.25rem;
}
.patient-avatar {
  width: 56px;
  height: 56px;
  border-radius: 50%;
  background: #1e293b;
  border: 2px solid #3b82f6;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.75rem;
}
.patient-title-row {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}
.patient-title-row h2 {
  font-family: var(--font-heading);
  font-size: 1.5rem;
  font-weight: 700;
}
.patient-history {
  font-size: 0.85rem;
  color: var(--text-secondary);
  margin-top: 0.2rem;
}
.patient-quick-stats {
  display: flex;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}
.stat-pill {
  background: var(--bg-main);
  border: 1px solid var(--border-subtle);
  padding: 0.45rem 0.85rem;
  border-radius: var(--radius-sm);
  font-size: 0.82rem;
}
.stat-pill .label {
  color: var(--text-muted);
  margin-right: 0.35rem;
}
.badge {
  padding: 0.25rem 0.65rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 600;
  display: inline-block;
}
.badge-age {
  background: #1e293b;
  color: #93c5fd;
}
.badge-normal, .badge-success {
  background: rgba(16, 185, 129, 0.15);
  color: #34d399;
  border: 1px solid rgba(16, 185, 129, 0.3);
}
.badge-warning {
  background: rgba(245, 158, 11, 0.15);
  color: #fbbf24;
  border: 1px solid rgba(245, 158, 11, 0.3);
}
.badge-emergency {
  background: rgba(239, 68, 68, 0.2);
  color: #f87171;
  border: 1px solid rgba(239, 68, 68, 0.4);
  animation: dangerBlink 1s infinite alternate;
}
}

```

```

.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  padding: 0.55rem 1.15rem;
  border-radius: var(--radius-sm);
  font-size: 0.85rem;
  font-weight: 600;
  cursor: pointer;
  transition: all var(--transition-fast);
  border: 1px solid transparent;
  font-family: var(--font-body);
}
.btn-primary {
  background: linear-gradient(135deg, #2563eb, #1d4ed8);
  color: #fff;
  box-shadow: 0 4px 12px rgba(37, 99, 235, 0.3);
}
.btn-primary:hover {
  background: linear-gradient(135deg, #3b82f6, #2563eb);
  transform: translateY(-1px);
}
.btn-success {
  background: linear-gradient(135deg, #10b981, #059669);
  color: #fff;
}
.btn-success:hover {
  background: linear-gradient(135deg, #34d399, #10b981);
}
.btn-outline {
  background: transparent;
  border-color: var(--border-subtle);
  color: var(--text-secondary);
}
.btn-outline:hover {
  background: var(--bg-card-hover);
  color: var(--text-primary);
  border-color: var(--border-highlight);
}
.btn-sm {
  padding: 0.35rem 0.75rem;
  font-size: 0.78rem;
}
.metrics-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.25rem;
}
.metric-card {
  background: var(--bg-card);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-md);
  padding: 1.35rem;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  transition: all var(--transition-fast);
  position: relative;
  overflow: hidden;
}
.metric-card::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 3px;
  background: transparent;
  transition: background var(--transition-fast);
}
.metric-card:hover {
  border-color: var(--border-highlight);
  transform: translateY(-2px);
  box-shadow: var(--shadow-card);
}
#cardHeartRate::before { background: linear-gradient(90deg, #f43f5e, #be123c); }
#cardSpO2::before { background: linear-gradient(90deg, #06b6d4, #0284c7); }
#cardTemp::before { background: linear-gradient(90deg, #f59e0b, #d97706); }
#cardMotion::before { background: linear-gradient(90deg, #10b981, #059669); }
.metric-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.metric-title {
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-secondary);
  text-transform: uppercase;
  letter-spacing: 0.04em;
}
.card-icon {
  font-size: 1.35rem;
}
.metric-body {
  margin: 1rem 0;
  display: flex;
  align-items: baseline;
  gap: 0.4rem;
}
.metric-value {
  font-family: var(--font-heading);
  font-size: 2.75rem;
  font-weight: 800;
  letter-spacing: -0.03em;
  color: #fff;
}
.metric-unit {
  font-size: 0.9rem;
  color: var(--text-muted);
  font-weight: 500;
}
}
.metric-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-top: 1px solid rgba(255, 255, 255, 0.05);
  padding-top: 0.65rem;
}
.threshold-tag {
  font-size: 0.75rem;
  color: var(--text-muted);
}
.visuals-split {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1.5rem;
}
@media (max-width: 1024px) {
  .visuals-split {
    grid-template-columns: 1fr;
  }
}
.panel-card {
  background: var(--bg-card);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-lg);
  padding: 1.35rem;
  display: flex;
  flex-direction: column;
  box-shadow: var(--shadow-card);
}
.panel-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1rem;
}
.panel-title {
  font-family: var(--font-heading);
  font-size: 1.15rem;
  font-weight: 700;
}
.panel-subtitle {
  font-size: 0.78rem;
  color: var(--text-secondary);
}
.live-dot-tag {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  font-size: 0.7rem;
  font-weight: 700;
  color: #f43f5e;
  background: rgba(244, 63, 94, 0.12);
  padding: 0.25rem 0.55rem;
  border-radius: 9999px;
  border: 1px solid rgba(244, 63, 94, 0.25);
}
.pulse-dot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: #f43f5e;
  animation: pulseGlow 1.2s infinite;
}
@keyframes pulseGlow {
  0% { transform: scale(0.9); opacity: 0.7; }
  50% { transform: scale(1.4); opacity: 1; box-shadow: 0 0 8px #f43f5e; }
  100% { transform: scale(0.9); opacity: 0.7; }
}
.chart-container {
  height: 280px;
  width: 100%;
}
.map-container {
  height: 280px;
  width: 100%;
  border-radius: var(--radius-sm);
  overflow: hidden;
  border: 1px solid var(--border-subtle);
}
.map-info-bar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.78rem;
  color: var(--text-secondary);
  margin-top: 0.75rem;
  padding-top: 0.5rem;
  border-top: 1px solid rgba(255, 255, 255, 0.05);
}
.tracker-split {
  display: grid;
  grid-template-columns: 1.35fr 0.85fr;
  gap: 1.5rem;
}
@media (max-width: 1024px) {
  .tracker-split {
    grid-template-columns: 1fr;
  }
}
.table-responsive {
  overflow-x: auto;
}
.data-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.82rem;
  text-align: left;
}
.data-table th {
  background: #0f1420;
  color: var(--text-secondary);
}

```

```

padding: 0.65rem 0.75rem;
font-weight: 600;
border-bottom: 1px solid var(--border-subtle);
}
.data-table td {
padding: 0.75rem 0.75rem;
border-bottom: 1px solid rgba(255, 255, 255, 0.04);
vertical-align: middle;
}
.data-table tbody tr:hover {
background: rgba(255, 255, 255, 0.02);
}

.dispatch-list {
display: flex;
flex-direction: column;
gap: 0.75rem;
max-height: 280px;
overflow-y: auto;
}
.dispatch-item {
background: var(--bg-main);
border: 1px solid var(--border-subtle);
border-left: 3px solid #ef4444;
border-radius: var(--radius-sm);
padding: 0.75rem;
font-size: 0.8rem;
}
.dispatch-meta {
display: flex;
justify-content: space-between;
margin-bottom: 0.35rem;
color: var(--text-muted);
font-size: 0.72rem;
}
.dispatch-channel {
font-weight: 700;
color: #38bdf8;
}
.empty-state {
text-align: center;
padding: 2.5rem 1rem;
color: var(--text-muted);
font-size: 0.82rem;
}
.simulator-bar {
position: fixed;
bottom: 0;
left: 0;
right: 0;
background: rgba(14, 20, 32, 0.95);
backdrop-filter: blur(14px);
border-top: 1px solid var(--border-subtle);
padding: 0.85rem 2rem;
z-index: 900;
box-shadow: 0 -5px 25px rgba(0, 0, 0, 0.6);
}
.sim-header {
display: flex;
justify-content: space-between;
align-items: center;
max-width: 1440px;
margin: 0 auto;
flex-wrap: wrap;
gap: 0.75rem;
}
.sim-title {
display: flex;
align-items: center;
gap: 0.75rem;
}
.sim-badge {
background: #0284c7;
color: #fff;
font-size: 0.7rem;
font-weight: 800;
padding: 0.25rem 0.55rem;
border-radius: 4px;
letter-spacing: 0.05em;
}
.sim-desc {
font-size: 0.8rem;
color: var(--text-secondary);
}
.sim-actions {
display: flex;
gap: 0.6rem;
flex-wrap: wrap;
}
.btn-simulator {
padding: 0.4rem 0.85rem;
font-size: 0.78rem;
border-radius: var(--radius-sm);
font-weight: 600;
background: var(--bg-card);
border: 1px solid var(--border-subtle);
color: var(--text-primary);
}
.btn-simulator:hover {
border-color: #38bdf8;
transform: translateY(-1px);
}
.btn-stream {
background: rgba(59, 130, 246, 0.15);
border-color: rgba(59, 130, 246, 0.4);
color: #60a5fa;
}
.btn-fall {
background: rgba(239, 68, 68, 0.2);
border-color: rgba(239, 68, 68, 0.5);
color: #f87171;
font-weight: 700;
}

.btn-fall:hover {
background: var(--emergency);
color: #fff;
box-shadow: 0 0 15px rgba(239, 68, 68, 0.6);
}
.btn-tachy { border-color: rgba(244, 63, 94, 0.4); color: #fb7185; }
.btn-spo2 { border-color: rgba(6, 182, 212, 0.4); color: #22d3ee; }
.btn-wander { border-color: rgba(245, 158, 11, 0.4); color: #fbbf24; }
.btn-reset { border-color: rgba(148, 163, 184, 0.3); color: var(--text-secondary); }
.modal-backdrop {
position: fixed;
inset: 0;
background: rgba(0, 0, 0, 0.85);
backdrop-filter: blur(8px);
display: flex;
align-items: center;
justify-content: center;
z-index: 2000;
padding: 1.5rem;
transition: opacity 0.25s ease;
}
.modal-backdrop.hidden {
display: none;
opacity: 0;
pointer-events: none;
}
.modal-card {
background: var(--bg-card);
border: 1px solid var(--border-highlight);
border-radius: var(--radius-lg);
max-width: 540px;
width: 100%;
padding: 1.75rem;
box-shadow: 0 25px 50px 12px rgba(0, 0, 0, 0.7);
animation: modalPop 0.25s cubic-bezier(0.175, 0.885, 0.32, 1.275);
}
@keyframes modalPop {
0% { transform: scale(0.9); opacity: 0; }
100% { transform: scale(1); opacity: 1; }
}
.emergency-modal {
border: 2px solid var(--emergency);
background: radial-gradient(circle at top, #260b0e, #130608);
box-shadow: 0 0 50px rgba(239, 68, 68, 0.5);
text-align: center;
}
.modal-banner {
margin-bottom: 1.5rem;
}
.siren-icon {
font-size: 3rem;
display: inline-block;
animation: sirenShake 0.6s infinite alternate ease-in-out;
}
@keyframes sirenShake {
0% { transform: rotate(-10deg) scale(1); }
100% { transform: rotate(10deg) scale(1.15); }
}
.modal-banner h2 {
font-family: var(--font-heading);
color: #fca5a5;
font-size: 1.75rem;
font-weight: 800;
letter-spacing: -0.02em;
margin-top: 0.5rem;
}
.modal-banner p {
color: #f87171;
font-size: 0.85rem;
font-weight: 500;
}
.countdown-circle-wrap {
position: relative;
width: 150px;
height: 150px;
margin: 1.25rem auto;
}
.countdown-svg {
transform: rotate(-90deg);
width: 100%;
height: 100%;
}
.countdown-bg {
fill: none;
stroke: rgba(255, 255, 255, 0.1);
stroke-width: 8;
}
.countdown-progress {
fill: none;
stroke: var(--emergency);
stroke-width: 8;
stroke-dasharray: 283;
stroke-dashoffset: 0;
stroke-linecap: round;
transition: stroke-dashoffset 1s linear;
}
.countdown-number {
position: absolute;
inset: 0;
display: flex;
align-items: center;
justify-content: center;
font-family: var(--font-heading);
font-size: 3.5rem;
font-weight: 800;
color: #fff;
text-shadow: 0 0 20px rgba(239, 68, 68, 0.8);
}
.countdown-details {
margin: 1rem 0 1.5rem;
}
.countdown-notice {

```

```

    font-size: 0.9rem;
    color: #fecaca;
    line-height: 1.4;
}
.countdown-warning-box {
    margin-top: 0.75rem;
    background: rgba(0, 0, 0, 0.4);
    border: 1px solid rgba(239, 68, 68, 0.4);
    border-radius: var(--radius-sm);
    padding: 0.65rem;
    font-size: 0.8rem;
    color: #fca5a5;
}
.btn-cancel-alert {
    width: 100%;
    padding: 1.1rem;
    font-size: 1.05rem;
    font-weight: 800;
    background: linear-gradient(135deg, #10b981, #059669);
    color: #fff;
    border: none;
    border-radius: var(--radius-md);
    cursor: pointer;
    box-shadow: 0 6px 20px rgba(16, 185, 129, 0.4);
    transition: all 0.2s ease;
}
.btn-cancel-alert:hover {
    background: linear-gradient(135deg, #34d399, #10b981);
    transform: translateY(-2px);
    box-shadow: 0 8px 25px rgba(16, 185, 129, 0.6);
}
.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-bottom: 1px solid var(--border-subtle);
    padding-bottom: 0.85rem;
    margin-bottom: 1.25rem;
}
.modal-header h3 {
    font-family: var(--font-heading);
    font-size: 1.25rem;
}
.modal-close {
    background: transparent;
    border: none;
    font-size: 1.5rem;
    color: var(--text-muted);
    cursor: pointer;
}
.modal-close:hover {
    color: #fff;
}
.form-row {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1rem;
    margin-bottom: 1rem;
}
.form-group {
    display: flex;
    flex-direction: column;
    gap: 0.35rem;
    margin-bottom: 1rem;
}
.form-group label {
    font-size: 0.8rem;
    color: var(--text-secondary);
    font-weight: 500;
}
.form-group input, .form-group textarea {
    background: var(--bg-main);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-sm);
    padding: 0.6rem 0.85rem;
    color: #fff;
    font-family: var(--font-body);
    font-size: 0.85rem;
    outline: none;
}
.form-group input:focus, .form-group textarea:focus {
    border-color: #3b82f6;
    box-shadow: 0 0 2px rgba(59, 130, 246, 0.2);
}
.modal-footer {
    display: flex;
    justify-content: flex-end;
    gap: 0.75rem;
    border-top: 1px solid var(--border-subtle);
    padding-top: 1rem;
    margin-top: 0.5rem;
}
.resolve-desc {
    font-size: 0.82rem;
    color: var(--text-secondary);
    margin-bottom: 1rem;
}

```

Server.py

```

import asyncio
import csv
import io
import json
import sqlite3
import time
from datetime import datetime, date
from pathlib import Path
from typing import List, Optional
from fastapi import FastAPI, HTTPException, WebSocket, WebSocketDisconnect, Response, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, StreamingResponse

```

```

from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field
app = FastAPI(title="Elderly Care Monitoring System (ECMS)", version="2.0.0")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
BASE_DIR = Path(__file__).resolve().parent
STATIC_DIR = BASE_DIR / "static"
DB_PATH = BASE_DIR / "ecms.db"
def get_db():
    conn = sqlite3.connect(str(DB_PATH), check_same_thread=False)
    conn.row_factory = sqlite3.Row
    return conn
def init_db():
    with get_db() as conn:
        cursor = conn.cursor()
        # UserAccount Table
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT UNIQUE NOT NULL,
                full_name TEXT NOT NULL,
                role TEXT NOT NULL, -- Doctor, Family_Caregiver, Administrator
                phone TEXT,
                email TEXT
            )
        """)
        # ElderlyProfile Table
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS elderly_profiles (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                user_id INTEGER,
                full_name TEXT NOT NULL,
                age INTEGER NOT NULL,
                gender TEXT,
                medical_history TEXT,
                emergency_contact TEXT,
                doctor_name TEXT DEFAULT 'Dr. Arvind Sharma',
                doctor_phone TEXT DEFAULT '+91-9876500002',
                address TEXT DEFAULT '142 Avinashi Road, Peelamedu, Coimbatore',
                FOREIGN KEY(user_id) REFERENCES users(id)
            )
        """)
        # Add any missing columns to elderly_profiles if upgrading from older version
        for col, col_type in [
            ("doctor_name", "TEXT DEFAULT 'Dr. Arvind Sharma'"),
            ("doctor_phone", "TEXT DEFAULT '+91-9876500002'"),
            ("address", "TEXT DEFAULT '142 Avinashi Road, Peelamedu, Coimbatore'")
        ]:
            try:
                cursor.execute(f"ALTER TABLE elderly_profiles ADD COLUMN {col} {col_type}")
            except sqlite3.OperationalError:
                pass
        # ThresholdConfig Table (1:1 with Patient)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS threshold_configs (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                patient_id INTEGER UNIQUE NOT NULL,
                min_hr INTEGER DEFAULT 55,
                max_hr INTEGER DEFAULT 105,
                min_spo2 REAL DEFAULT 92.0,
                min_temp REAL DEFAULT 36.0,
                max_temp REAL DEFAULT 37.8,
                safe_lat REAL DEFAULT 11.0168,
                safe_lng REAL DEFAULT 76.9558,
                safe_radius_meters REAL DEFAULT 250.0,
                FOREIGN KEY(patient_id) REFERENCES elderly_profiles(id)
            )
        """)
        # HealthMetricLog Table (Time-Series Store)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS health_metrics (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                patient_id INTEGER NOT NULL,
                timestamp TEXT NOT NULL,
                heart_rate INTEGER NOT NULL,
                spo2 REAL NOT NULL,
                temperature REAL NOT NULL,
                steps INTEGER DEFAULT 0,
                latitude REAL NOT NULL,
                longitude REAL NOT NULL,
                motion_vector REAL DEFAULT 1.0,
                FOREIGN KEY(patient_id) REFERENCES elderly_profiles(id)
            )
        """)
        # IncidentAlert Table
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS incident_alerts (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                patient_id INTEGER NOT NULL,
                alert_type TEXT NOT NULL, -- FALL_DETECTED, VITAL_ANOMALY,
                GEOFENCE_BREACH, EMERGENCY_SOS
                severity TEXT NOT NULL, -- Warning, Emergency
                message TEXT NOT NULL,
                status TEXT NOT NULL, -- Verifying_Countdown, Emergency, Resolved, Cancelled
                doctor_notes TEXT,
                created_at TEXT NOT NULL,
                resolved_at TEXT,
                FOREIGN KEY(patient_id) REFERENCES elderly_profiles(id)
            )
        """)
        # DispatchLog Table (Emergency notification records: SMS, Email, Push)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS dispatch_logs (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                alert_id INTEGER,
                channel TEXT NOT NULL, -- SMS, EMAIL, PUSH, WHATSAPP
                recipient TEXT NOT NULL,
                payload TEXT NOT NULL,

```

```

        dispatched_at TEXT NOT NULL,
        status TEXT DEFAULT 'Delivered'
    )
)
# Medications Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS medications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    dosage TEXT NOT NULL,
    time TEXT NOT NULL, -- e.g. "08:00 AM"
    instructions TEXT,
    status TEXT DEFAULT 'pending', -- 'pending', 'taken', 'missed'
    last_taken_at TEXT,
    FOREIGN KEY(patient_id) REFERENCES elderly_profiles(id)
)
)
# Daily Wellness & Caregiver Journal Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS daily_wellness_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER NOT NULL,
    date TEXT NOT NULL,
    water_glasses INTEGER DEFAULT 4,
    meals_logged TEXT DEFAULT '{"breakfast": true, "lunch": false, "dinner": false}',
    sleep_hours REAL DEFAULT 7.5,
    mood TEXT DEFAULT 'Calm',
    notes TEXT DEFAULT 'Resting comfortably; vitals stable.',
    FOREIGN KEY(patient_id) REFERENCES elderly_profiles(id)
)
)
conn.commit()
# Seed initial users if empty
cursor.execute("SELECT COUNT(*) FROM users")
if cursor.fetchone()[0] == 0:
    cursor.execute("""
INSERT INTO users (username, full_name, role, phone, email) VALUES
('admin', 'Kathiravan A', 'Administrator', '+91-9876500001', 'admin@ecms-cit.edu'),
('dr_sharma', 'Dr. Arvind Sharma', 'Doctor', '+91-9876500002', 'dr.sharma@hospital.org'),
('caregiver_priya', 'Priya K (Daughter)', 'Family_Caregiver', '+91-9876500003',
'priya.k@gmail.com')
)
)
    conn.commit()
# Seed elderly profiles if only 0 or 1 exists
cursor.execute("SELECT COUNT(*) FROM elderly_profiles")
patient_count = cursor.fetchone()[0]
if patient_count == 0:
    # Seed Patient 1
    cursor.execute("""
INSERT INTO elderly_profiles (user_id, full_name, age, gender, medical_history,
emergency_contact, doctor_name, doctor_phone, address) VALUES
(3, 'Robert Jenkins', 78, 'Male', 'Hypertension, Mild Cardiac Arrhythmia, Osteoporosis',
'+91-9876500003', 'Dr. Arvind Sharma (Cardiologist)', '+91-9876500002', '142 Avinashi Road,
Peelamedu, Coimbatore')
)
)
    p1_id = cursor.lastrowid
    cursor.execute("""
INSERT INTO threshold_configs (patient_id, min_hr, max_hr, min_spo2, min_temp,
max_temp, safe_lat, safe_lng, safe_radius_meters)
VALUES (?, 55, 105, 92.0, 36.0, 37.8, 11.0168, 76.9558, 250.0)
)
)
    conn.commit()
# Seed multi-patient demonstration profiles (Patient 2 & 3) if not present
cursor.execute("SELECT COUNT(*) FROM elderly_profiles")
if cursor.fetchone()[0] < 3:
    # Patient 2: Margaret Smith
    cursor.execute("""
INSERT INTO elderly_profiles (user_id, full_name, age, gender, medical_history,
emergency_contact, doctor_name, doctor_phone, address) VALUES
(3, 'Margaret Smith', 82, 'Female', 'Early-Stage Alzheimer's, Osteoarthritis, High Fall &
Wander Risk', '+91-9876500004 (David Smith, Son)', 'Dr. Meenakshi Sundaram (Geriatrician)', '+91-
9876500008', '28 Race Course Road, Coimbatore')
)
)
    p2_id = cursor.lastrowid
    cursor.execute("""
INSERT INTO threshold_configs (patient_id, min_hr, max_hr, min_spo2, min_temp,
max_temp, safe_lat, safe_lng, safe_radius_meters)
VALUES (?, 58, 100, 93.0, 36.0, 37.7, 11.0055, 76.9680, 200.0)
)
)
    conn.commit()
# Patient 3: Sundar Rajan
cursor.execute("""
INSERT INTO elderly_profiles (user_id, full_name, age, gender, medical_history,
emergency_contact, doctor_name, doctor_phone, address) VALUES
(3, 'Sundar Rajan', 74, 'Male', 'Type 2 Diabetes, Diabetic Neuropathy, Post-Bypass Cardiac
Rehab', '+91-9876500005 (Ananya Rajan, Wife)', 'Dr. Rajesh Kannan (Endocrinologist)', '+91-
9876500009', '85 Gandhipuram 4th Cross, Coimbatore')
)
)
    p3_id = cursor.lastrowid
    cursor.execute("""
INSERT INTO threshold_configs (patient_id, min_hr, max_hr, min_spo2, min_temp,
max_temp, safe_lat, safe_lng, safe_radius_meters)
VALUES (?, 52, 108, 91.5, 36.0, 37.9, 11.0185, 76.9642, 300.0)
)
)
    conn.commit()

# Seed baseline metrics for any patient lacking health_metrics
cursor.execute("SELECT id FROM elderly_profiles")
patients = cursor.fetchall()
now = time.time()
for p in patients:
    pid = p["id"]
    cursor.execute("SELECT COUNT(*) FROM health_metrics WHERE patient_id = ?", (pid,))
    if cursor.fetchone()[0] == 0:
        for i in range(12, 0, -1):
            t_str = datetime.fromtimestamp(now - i * 30).strftime("%Y-%m-%d %H:%M:%S")
            base_lat = 11.0168 if pid == 1 else (11.0055 if pid == 2 else 11.0185)
            base_lng = 76.9558 if pid == 1 else (76.9680 if pid == 2 else 76.9642)
            cursor.execute("""
INSERT INTO health_metrics (patient_id, timestamp, heart_rate, spo2, temperature,
steps, latitude, longitude, motion_vector)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
)
)
            conn.commit()
            (pid, t_str, 72 + ((i + pid) % 6), 97.5 - ((i % 3) * 0.4), 36.6, 2100 + i * 35,
            base_lat + (i * 0.00002), base_lng - (i * 0.00002), 0.98))

```

```

        conn.commit()
# Seed default medications if medications table is empty
cursor.execute("SELECT COUNT(*) FROM medications")
if cursor.fetchone()[0] == 0:
    today_str = date.today().isoformat()
    # Patient 1 Medications
    cursor.execute("""
INSERT INTO medications (patient_id, name, dosage, time, instructions, status,
last_taken_at) VALUES
(1, 'Amlodipine Besylate', '5 mg', '08:00 AM', 'Blood Pressure — Take after breakfast with
a full glass of water', 'taken', ?),
(1, 'Aspirin (Cardio)', '75 mg', '01:00 PM', 'Blood Thinner — Take after lunch to prevent
gastric irritation', 'pending', NULL),
(1, 'Atorvastatin', '20 mg', '08:00 PM', 'Cholesterol / Heart Health — Take before bedtime',
'pending', NULL)
)
)
    conn.commit()
# Patient 2 Medications
cursor.execute("""
INSERT INTO medications (patient_id, name, dosage, time, instructions, status,
last_taken_at) VALUES
(2, 'Donepezil Hydrochloride', '10 mg', '09:00 AM', 'Cognitive Support — Take with
morning meal', 'taken', ?),
(2, 'Calcium Carbonate + D3', '500 mg', '02:00 PM', 'Bone Health / Osteoarthritis — Take
with milk or meal', 'pending', NULL),
(2, 'Melatonin', '3 mg', '09:00 PM', 'Sleep Regulation — Take 30 minutes before sleep',
'pending', NULL)
)
)
    conn.commit()
# Patient 3 Medications
cursor.execute("""
INSERT INTO medications (patient_id, name, dosage, time, instructions, status,
last_taken_at) VALUES
(3, 'Metformin Extended Release', '500 mg', '08:30 AM', 'Blood Glucose Control — Take
with breakfast', 'taken', ?),
(3, 'Losartan Potassium', '50 mg', '01:30 PM', 'Renal Protection & BP — Take with lunch',
'taken', ?),
(3, 'Glimepiride', '2 mg', '07:30 PM', 'Diabetes Management — Take immediately before
dinner', 'pending', NULL)
)
)
    conn.commit()
# Seed daily wellness logs if empty
cursor.execute("SELECT COUNT(*) FROM daily_wellness_logs")
if cursor.fetchone()[0] == 0:
    today_str = date.today().isoformat()
    cursor.execute("""
INSERT INTO daily_wellness_logs (patient_id, date, water_glasses, meals_logged,
sleep_hours, mood, notes) VALUES
(1, ?, 5, '{"breakfast": true, "lunch": false, "dinner": false}', 7.5, 'Calm', 'Resting well in
morning. Morning walk completed in garden.'),
(2, ?, 4, '{"breakfast": true, "lunch": true, "dinner": false}', 6.8, 'Happy', 'Good memory recall
today. Participated in art therapy.'),
(3, ?, 6, '{"breakfast": true, "lunch": true, "dinner": false}', 8.0, 'Calm', 'Blood sugar fasting
114 mg/dL. Normal sensation in feet.')
)
)
    conn.commit()

init_db()
class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []
    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)
    def disconnect(self, websocket: WebSocket):
        if websocket in self.active_connections:
            self.active_connections.remove(websocket)

    async def broadcast(self, message: dict):
        for connection in list(self.active_connections):
            try:
                await connection.send_json(message)
            except Exception:
                self.disconnect(connection)
manager = ConnectionManager()
active_countdown_task: Optional[asyncio.Task] = None
current_verifying_alert_id: Optional[int] = None
def haversine_distance(lat1, lon1, lat2, lon2):
    """Calculates approximate distance between two GPS coordinates in meters."""
    from math import radians, cos, sin, asin, sqrt
    r = 6371000 # radius of Earth in meters
    d_lat = radians(lat2 - lat1)
    d_lon = radians(lon2 - lon1)
    a = sin(d_lat / 2)**2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(d_lon / 2)**2
    c = 2 * asin(sqrt(a))
    return r * c
async def run_fall_countdown(alert_id: int, patient_id: int):
    """15-second non-blocking countdown safeguard. If not cancelled, triggers emergency dispatch."""
    global current_verifying_alert_id
    current_verifying_alert_id = alert_id
    try:
        for remaining in range(15, -1, -1):
            with get_db() as conn:
                cur = conn.cursor()
                cur.execute("SELECT status FROM incident_alerts WHERE id = ?", (alert_id,))
                row = cur.fetchone()
                if not row or row["status"] != "Verifying_Countdown":
                    # Cancelled by user!
                    return
            await manager.broadcast({
                "type": "COUNTDOWN_TICK",
                "alert_id": alert_id,
                "patient_id": patient_id,
                "remaining_seconds": remaining
            })
            if remaining == 0:
                # Countdown expired -> Transition to EMERGENCY & Dispatch!
                execute_emergency_dispatch(alert_id, patient_id, reason="Fall impact verified (15s
                timeout expired)")
                return
            await asyncio.sleep(1)
    except asyncio.CancelledError:
        pass
    finally:
        current_verifying_alert_id = None
def execute_emergency_dispatch(alert_id: int, patient_id: int, reason: str = "Fall impact verified"):

```

```

"""Executes multi-channel notification dispatch (SMS, Email, Push, WhatsApp) upon confirmed
emergency."""
now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with get_db() as conn:
    cur = conn.cursor()
    cur.execute("""
        UPDATE incident_alerts
        SET status = 'Emergency', severity = 'Emergency'
        WHERE id = ?
        """, (alert_id,))

    cur.execute("SELECT full_name, emergency_contact, doctor_name, doctor_phone FROM
elderly_profiles WHERE id = ?", (patient_id,))
    pat = cur.fetchone()
    cur.execute("SELECT latitude, longitude, heart_rate, spo2, temperature FROM health_metrics
WHERE patient_id = ? ORDER BY id DESC LIMIT 1", (patient_id,))
    metric = cur.fetchone()
    lat = metric["latitude"] if metric else 11.0168
    lng = metric["longitude"] if metric else 76.9558
    hr = metric["heart_rate"] if metric else 75
    spo2 = metric["spo2"] if metric else 98.0
    contact = pat["emergency_contact"] if pat else "+91-9876500003"
    patient_name = pat["full_name"] if pat else "Robert Jenkins"
    maps_url = f"https://maps.google.com/?q={lat:.5f},{lng:.5f}"
    sms_body = f"CRITICAL EMERGENCY: {reason} for {patient_name}! Vitals: HR={hr}bpm,
SpO2={spo2}%. Location: {maps_url}. Immediate assistance dispatched."
    email_body = f"Subject: URGENT Medical Emergency Alert - {patient_name}\n\nEmergency
Trigger: {reason}\nVitals at Incident: Heart Rate = {hr} bpm, SpO2 = {spo2}%, Temp =
{metric['temperature']} if metric else 36.6°C\nReal-Time GPS Coordinates: ({lat:.5f},
{lng:.5f})\nLive Map: {maps_url}\nDispatcher Action: Automated notification dispatched to primary
caregiver ({contact}) and ambulance standby."
    whatsapp_body = f"🚑 *EMERGENCY ALERT - ECMS CarePortal*\n*Patient:*
{patient_name}\n*Alert:* {reason}\n*Vitals:* HR: {hr} bpm | SpO2: {spo2}%\n*Location:*
{maps_url}\n*Time:* {now_str}"
    # Multi-channel logs
    cur.execute("INSERT INTO dispatch_logs (alert_id, channel, recipient, payload, dispatched_at)
VALUES (?, 'SMS', ?, ?, ?)",
        (alert_id, contact, sms_body, now_str))
    cur.execute("INSERT INTO dispatch_logs (alert_id, channel, recipient, payload, dispatched_at)
VALUES (?, 'EMAIL', ?, ?, ?)",
        (alert_id, "caregiver@ecms-cit.edu", email_body, now_str))
    cur.execute("INSERT INTO dispatch_logs (alert_id, channel, recipient, payload, dispatched_at)
VALUES (?, 'PUSH', ?, ?, ?)",
        (alert_id, f"Caregiver App [{patient_name}]", f"EMERGENCY: {reason} for
{patient_name}", now_str))
    cur.execute("INSERT INTO dispatch_logs (alert_id, channel, recipient, payload, dispatched_at)
VALUES (?, 'WHATSAPP', ?, ?, ?)",
        (alert_id, contact, whatsapp_body, now_str))
    conn.commit()
    asyncio.create_task(manager.broadcast({
        "type": "EMERGENCY_CONFIRMED",
        "alert_id": alert_id,
        "patient_id": patient_id,
        "message": f"Critical emergency confirmed for {patient_name}! Multi-channel alerts dispatched
to family and doctors."
    }))
class MetricPayload(BaseModel):
    patient_id: int = 1
    heart_rate: int = Field(..., ge=20, le=240)
    spo2: float = Field(..., ge=50.0, le=100.0)
    temperature: float = Field(..., ge=30.0, le=45.0)
    steps: int = Field(default=0, ge=0)
    latitude: float = Field(...)
    longitude: float = Field(...)
    motion_vector: float = Field(default=1.0) # > 3.2g indicates fall deceleration
class ThresholdUpdate(BaseModel):
    min_hr: int
    max_hr: int
    min_spo2: float
    min_temp: float
    max_temp: float
    safe_lat: float
    safe_lng: float
    safe_radius_meters: float
class ResolveAlertPayload(BaseModel):
    doctor_notes: str
class MedicationCreate(BaseModel):
    name: str
    dosage: str
    time: str
    instructions: Optional[str] = ""
class WellnessUpdate(BaseModel):
    water_glasses: Optional[int] = None
    meals_logged: Optional[str] = None
    sleep_hours: Optional[float] = None
    mood: Optional[str] = None
    notes: Optional[str] = None
class PatientCreate(BaseModel):
    full_name: str
    age: int
    gender: str
    medical_history: str
    emergency_contact: str
    doctor_name: Optional[str] = "Dr. Arvind Sharma"
    doctor_phone: Optional[str] = "+91-9876500002"
    address: Optional[str] = "Coimbatore, Tamil Nadu"
    safe_lat: Optional[float] = 11.0168
    safe_lng: Optional[float] = 76.9558
    safe_radius_meters: Optional[float] = 250.0
class PatientUpdate(BaseModel):
    full_name: str
    age: int
    gender: str
    medical_history: str
    emergency_contact: str
    doctor_name: str
    doctor_phone: str
    address: str
@app.get("/api/patients")
def list_all_patients():
    """Returns list of all monitored senior citizens with summary status."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM elderly_profiles ORDER BY id ASC")

```

```

patients = [dict(r) for r in cur.fetchall()]
for p in patients:
    pid = p["id"]
    # Latest metric
    cur.execute("SELECT * FROM health_metrics WHERE patient_id = ? ORDER BY id DESC
LIMIT 1", (pid,))
    metric = cur.fetchone()
    p["latest_metric"] = dict(metric) if metric else None
    # Thresholds
    cur.execute("SELECT * FROM threshold_configs WHERE patient_id = ?", (pid,))
    thresh = cur.fetchone()
    p["thresholds"] = dict(thresh) if thresh else {}
    # Active alerts
    cur.execute("SELECT * FROM incident_alerts WHERE patient_id = ? AND status IN
('Verifying_Countdown', 'Emergency') ORDER BY id DESC LIMIT 1", (pid,))
    active_alert = cur.fetchone()
    p["active_alert"] = dict(active_alert) if active_alert else None
    # Pending meds count
    cur.execute("SELECT COUNT(*) FROM medications WHERE patient_id = ? AND status =
'pending'", (pid,))
    p["pending_meds_count"] = cur.fetchone()[0]
    # Overall Status
    if active_alert:
        p["status_code"] = "emergency" if active_alert["status"] == "Emergency" else "countdown"
    elif metric and thresh:
        if (metric["heart_rate"] > thresh["max_hr"] or
            metric["heart_rate"] < thresh["min_hr"] or
            metric["spo2"] < thresh["min_spo2"]):
            p["status_code"] = "warning"
        else:
            p["status_code"] = "normal"
    else:
        p["status_code"] = "normal"
    return patients
@app.get("/api/patient")
def get_patient_profile(patient_id: int = 1):
    """Retrieves full profile, vitals, active alert, medications, and wellness for specific patient."""
    today_str = date.today().isoformat()
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM elderly_profiles WHERE id = ?", (patient_id,))
        patient = cur.fetchone()
        if not patient:
            raise HTTPException(status_code=404, detail="Patient profile not found")
        cur.execute("SELECT * FROM threshold_configs WHERE patient_id = ?", (patient_id,))
        thresholds = cur.fetchone()
        cur.execute("SELECT * FROM health_metrics WHERE patient_id = ? ORDER BY id DESC
LIMIT 1", (patient_id,))
        latest_metric = cur.fetchone()
        cur.execute("SELECT * FROM incident_alerts WHERE patient_id = ? AND status IN
('Verifying_Countdown', 'Emergency') ORDER BY id DESC LIMIT 1", (patient_id,))
        active_alert = cur.fetchone()

        cur.execute("SELECT * FROM medications WHERE patient_id = ? ORDER BY id ASC",
(patient_id,))
        medications = [dict(r) for r in cur.fetchall()]
        cur.execute("SELECT * FROM daily_wellness_logs WHERE patient_id = ? AND date = ?
LIMIT 1", (patient_id, today_str))
        wellness = cur.fetchone()
        if not wellness:
            cur.execute("""
                INSERT INTO daily_wellness_logs (patient_id, date, water_glasses, meals_logged,
sleep_hours, mood, notes)
                VALUES (?, ?, 4, {'breakfast': false, 'lunch': false, 'dinner': false}, 7.0, 'Calm', 'Day
started.'
            """, (patient_id, today_str))
            conn.commit()
        cur.execute("SELECT * FROM daily_wellness_logs WHERE patient_id = ? AND date = ?
LIMIT 1", (patient_id, today_str))
        wellness = cur.fetchone()
        return {
            "profile": dict(patient),
            "thresholds": dict(thresholds) if thresholds else {},
            "latest_metric": dict(latest_metric) if latest_metric else {},
            "active_alert": dict(active_alert) if active_alert else None,
            "medications": medications,
            "wellness": dict(wellness) if wellness else {}
        }
@app.post("/api/patients")
async def create_new_patient(payload: PatientCreate):
    """Registers a new elderly citizen into the care system."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            INSERT INTO elderly_profiles (user_id, full_name, age, gender, medical_history,
emergency_contact, doctor_name, doctor_phone, address)
            VALUES (3, ?, ?, ?, ?, ?, ?, ?)
        """, (payload.full_name, payload.age, payload.gender, payload.medical_history,
payload.emergency_contact,
payload.doctor_name, payload.doctor_phone, payload.address))
        pid = cur.lastrowid
        cur.execute("""
            INSERT INTO threshold_configs (patient_id, min_hr, max_hr, min_spo2, min_temp,
max_temp, safe_lat, safe_lng, safe_radius_meters)
            VALUES (?, 55, 105, 92.0, 36.0, 37.8, ?, ?, ?)
        """, (pid, payload.safe_lat, payload.safe_lng, payload.safe_radius_meters))
        # Seed initial metric
        now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        cur.execute("""
            INSERT INTO health_metrics (patient_id, timestamp, heart_rate, spo2, temperature, steps,
latitude, longitude, motion_vector)
            VALUES (?, ?, 74, 98.0, 36.6, 1200, ?, ?, 0.98)
        """, (pid, now_str, payload.safe_lat, payload.safe_lng))
        # Initialize wellness
        cur.execute("""
            INSERT INTO daily_wellness_logs (patient_id, date, water_glasses, meals_logged,
sleep_hours, mood, notes)
            VALUES (?, ?, 4, {'breakfast': false, 'lunch': false, 'dinner': false}, 7.5, 'Calm', 'Initial
intake recorded.'
        """, (pid, date.today().isoformat()))
        conn.commit()
        await manager.broadcast({"type": "PATIENT_UPDATED", "patient_id": pid})
        return {"status": "success", "patient_id": pid, "message": f"Patient profile for {payload.full_name}
created successfully."}

```

```

@app.put("/api/patients/{patient_id}")
async def update_patient_profile(patient_id: int, payload: PatientUpdate):
    """Updates senior's demographics, medical history, and contact coordinates."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            UPDATE elderly_profiles
            SET full_name=?, age=?, gender=?, medical_history=?, emergency_contact=?,
            doctor_name=?, doctor_phone=?, address=?
            WHERE id=?
            """, (payload.full_name, payload.age, payload.gender, payload.medical_history,
            payload.emergency_contact,
            payload.doctor_name, payload.doctor_phone, payload.address, patient_id))
        conn.commit()
        await manager.broadcast({"type": "PATIENT_UPDATED", "patient_id": patient_id})
        return {"status": "success", "message": "Patient profile updated successfully."}
@app.get("/api/patients/{patient_id}/medications")
def get_patient_medications(patient_id: int):
    """Fetches medication schedule and computes daily adherence score."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM medications WHERE patient_id = ? ORDER BY id ASC",
            (patient_id,))
        meds = [dict(r) for r in cur.fetchall()]
        total = len(meds)
        taken = sum(1 for m in meds if m["status"] == "taken")
        adherence_rate = round((taken / total * 100), 1) if total > 0 else 100.0
        return {
            "patient_id": patient_id,
            "total_medications": total,
            "taken_medications": taken,
            "adherence_rate": adherence_rate,
            "medications": meds
        }
@app.post("/api/patients/{patient_id}/medications")
async def add_patient_medication(patient_id: int, payload: MedicationCreate):
    """Adds a new prescription schedule for senior."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            INSERT INTO medications (patient_id, name, dosage, time, instructions, status)
            VALUES (?, ?, ?, ?, ?, 'pending')
            """, (patient_id, payload.name, payload.dosage, payload.time, payload.instructions))
        med_id = cur.lastrowid
        conn.commit()
        await manager.broadcast({"type": "MEDICATION_UPDATED", "patient_id": patient_id})
        return {"status": "success", "medication_id": med_id, "message": "Medication added to schedule."}
@app.post("/api/medications/{med_id}/take")
async def toggle_medication_taken(med_id: int):
    """Toggles medication taken state and logs exact timestamp."""
    now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT patient_id, status FROM medications WHERE id = ?", (med_id,))
        row = cur.fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Medication not found")
        patient_id = row["patient_id"]
        new_status = "pending" if row["status"] == "taken" else "taken"
        taken_at = now_str if new_status == "taken" else None
        cur.execute("UPDATE medications SET status = ?, last_taken_at = ? WHERE id = ?",
            (new_status, taken_at, med_id))
        conn.commit()
        await manager.broadcast({
            "type": "MEDICATION_UPDATED",
            "patient_id": patient_id,
            "med_id": med_id,
            "status": new_status
        })
        return {"status": "success", "new_status": new_status, "last_taken_at": taken_at}
@app.delete("/api/medications/{med_id}")
async def delete_medication(med_id: int):
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT patient_id FROM medications WHERE id = ?", (med_id,))
        row = cur.fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Medication not found")
        patient_id = row["patient_id"]
        cur.execute("DELETE FROM medications WHERE id = ?", (med_id,))
        conn.commit()
        await manager.broadcast({"type": "MEDICATION_UPDATED", "patient_id": patient_id})
        return {"status": "success", "message": "Medication removed."}
@app.get("/api/patients/{patient_id}/wellness")
def get_patient_wellness(patient_id: int):
    today_str = date.today().isoformat()
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM daily_wellness_logs WHERE patient_id = ? AND date = ?",
            (patient_id, today_str))
        row = cur.fetchone()
        if not row:
            cur.execute("""
                INSERT INTO daily_wellness_logs (patient_id, date, water_glasses, meals_logged,
                sleep_hours, mood, notes)
                VALUES (?, ?, 4, {'breakfast': false, 'lunch': false, 'dinner': false}, 7.0, 'Calm', 'Day
                initialized.')
                """, (patient_id, today_str))
            conn.commit()
            cur.execute("SELECT * FROM daily_wellness_logs WHERE patient_id = ? AND date = ?",
                (patient_id, today_str))
            row = cur.fetchone()
            return dict(row)
    @app.post("/api/patients/{patient_id}/wellness")
    async def update_patient_wellness(patient_id: int, payload: WellnessUpdate):
        today_str = date.today().isoformat()
        with get_db() as conn:
            cur = conn.cursor()
            cur.execute("SELECT * FROM daily_wellness_logs WHERE patient_id = ? AND date = ?",
                (patient_id, today_str))
            existing = cur.fetchone()
            water = payload.water_glasses if payload.water_glasses is not None else

```

```

(existing["water_glasses"] if existing else 4)
            meals = payload.meals_logged if payload.meals_logged is not None else
            (existing["meals_logged"] if existing else {'breakfast': false, 'lunch': false, 'dinner': false})
            sleep = payload.sleep_hours if payload.sleep_hours is not None else (existing["sleep_hours"] if
            existing else 7.0)
            mood = payload.mood if payload.mood is not None else (existing["mood"] if existing else 'Calm')
            notes = payload.notes if payload.notes is not None else (existing["notes"] if existing else "")
            if existing:
                cur.execute("""
                    UPDATE daily_wellness_logs
                    SET water_glasses=?, meals_logged=?, sleep_hours=?, mood=?, notes=?
                    WHERE id=?
                    """, (water, meals, sleep, mood, notes, existing["id"]))
            else:
                cur.execute("""
                    INSERT INTO daily_wellness_logs (patient_id, date, water_glasses, meals_logged,
                    sleep_hours, mood, notes)
                    VALUES (?, ?, ?, ?, ?, ?, ?)
                    """, (patient_id, today_str, water, meals, sleep, mood, notes))
                conn.commit()
                await manager.broadcast({"type": "WELLNESS_UPDATED", "patient_id": patient_id})
                return {"status": "success", "message": "Wellness journal updated."}
@app.post("/api/patients/{patient_id}/sos")
async def trigger_emergency_sos(patient_id: int):
    """Emergency SOS button pressed by senior or family caregiver."""
    now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT full_name, emergency_contact, doctor_name, doctor_phone FROM
            elderly_profiles WHERE id = ?", (patient_id,))
        pat = cur.fetchone()
        if not pat:
            raise HTTPException(status_code=404, detail="Patient profile not found")
        cur.execute("SELECT latitude, longitude, heart_rate, spo2 FROM health_metrics WHERE
            patient_id = ? ORDER BY id DESC LIMIT 1", (patient_id,))
        met = cur.fetchone()
        lat = met["latitude"] if met else 11.0168
        lng = met["longitude"] if met else 76.9558
        msg = f"EMERGENCY SOS: Panic button manually triggered for {pat['full_name']}! Immediate
        assistance required."
        cur.execute("""
            INSERT INTO incident_alerts (patient_id, alert_type, severity, message, status, created_at)
            VALUES (?, 'EMERGENCY_SOS', 'Emergency', ?, 'Emergency', ?)
            """, (patient_id, msg, now_str))
        alert_id = cur.lastrowid
        conn.commit()
        execute_emergency_dispatch(alert_id, patient_id, reason="Manual Emergency SOS Panic
        Trigger")
        # WhatsApp pre-filled message generator
        maps_url = f"https://maps.google.com/?q={lat:.5f},{lng:.5f}"
        wa_text = f"EMERGENCY SOS ALERT for {pat['full_name']}! Immediate help needed. Location:
        {maps_url}"
        clean_phone = pat["emergency_contact"].replace("+", "").replace("-", "").replace(" ",
            "").split("")[0]
        return {
            "status": "success",
            "alert_id": alert_id,
            "message": f"Emergency SOS initiated for {pat['full_name']}.",
            "whatsapp_url": f"https://wa.me/{clean_phone}?text={wa_text}"
        }
@app.post("/api/thresholds")
def update_thresholds(payload: ThresholdUpdate, patient_id: int = 1):
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            UPDATE threshold_configs
            SET min_hr=?, max_hr=?, min_spo2=?, min_temp=?, max_temp=?, safe_lat=?, safe_lng=?,
            safe_radius_meters=?
            WHERE patient_id=?
            """, (payload.min_hr, payload.max_hr, payload.min_spo2, payload.min_temp,
            payload.max_temp,
            payload.safe_lat, payload.safe_lng, payload.safe_radius_meters, patient_id))
        conn.commit()
        return {"status": "success", "message": "Thresholds updated successfully"}
@app.get("/api/history")
def get_metric_history(patient_id: int = 1, limit: int = 25):
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT id, timestamp, heart_rate, spo2, temperature, steps, latitude, longitude,
            motion_vector
            FROM health_metrics
            WHERE patient_id = ?
            ORDER BY id DESC LIMIT ?
            """, (patient_id, limit))
        rows = cur.fetchall()
        return [dict(r) for r in reversed(rows)]
@app.get("/api/alerts")
def get_incident_alerts(patient_id: int = 1):
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT * FROM incident_alerts
            WHERE patient_id = ?
            ORDER BY id DESC LIMIT 30
            """, (patient_id,))
        return [dict(r) for r in cur.fetchall()]
@app.get("/api/dispatch-logs")
def get_dispatch_logs():
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM dispatch_logs ORDER BY id DESC LIMIT 30")
        return [dict(r) for r in cur.fetchall()]
@app.post("/api/metrics")
async def ingest_metric(payload: MetricPayload):
    """Real-Time Vital Ingestion Endpoint (REQ-1, 2, 3, 4, 5)."""
    global active_countdown_task
    now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with get_db() as conn:
        cur = conn.cursor()
        # 1. Fetch threshold configuration
        cur.execute("SELECT * FROM threshold_configs WHERE patient_id = ?",
            (payload.patient_id,))

```

```

cfg = cur.fetchone()
# 2. Store time-series log
cur.execute("""
INSERT INTO health_metrics (patient_id, timestamp, heart_rate, spo2, temperature, steps,
latitude, longitude, motion_vector)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
""", (payload.patient_id, now_str, payload.heart_rate, payload.spo2, payload.temperature,
payload.steps, payload.latitude, payload.longitude, payload.motion_vector))
log_id = cur.lastrowid
anomalies = []
if cfg:
    if payload.heart_rate > cfg["max_hr"]:
        anomalies.append(f"Tachycardia detected: Heart rate {payload.heart_rate} bpm exceeds
maximum {cfg['max_hr']} bpm")
    elif payload.heart_rate < cfg["min_hr"]:
        anomalies.append(f"Bradycardia detected: Heart rate {payload.heart_rate} bpm below
minimum {cfg['min_hr']} bpm")
    if payload.spo2 < cfg["min_spo2"]:
        anomalies.append(f"Hypoxemia warning: SpO2 level {payload.spo2}% dropped below safe
limit {cfg['min_spo2']}%")
    if payload.temperature > cfg["max_temp"]:
        anomalies.append(f"High fever detected: Body temperature {payload.temperature:.1f}°C
exceeds {cfg['max_temp']}°C")
    dist = haversine_distance(cfg["safe_lat"], cfg["safe_lng"], payload.latitude,
payload.longitude)
    if dist > cfg["safe_radius_meters"]:
        anomalies.append(f"Geo-fence breach: Senior is {dist:.1f}m away from safe center (Limit:
{cfg['safe_radius_meters']}m)")
    is_fall = payload.motion_vector >= 3.2
    alert_id = None
    if is_fall:
        cur.execute("""
INSERT INTO incident_alerts (patient_id, alert_type, severity, message, status, created_at)
VALUES (?, 'FALL_DETECTED', 'Warning', 'High deceleration impact detected! 15s
verification countdown started.', 'Verifying_Countdown', ?)
""", (payload.patient_id, now_str))
        alert_id = cur.lastrowid
        conn.commit()
        if active_countdown_task and not active_countdown_task.done():
            active_countdown_task.cancel()
            active_countdown_task = asyncio.create_task(run_fall_countdown(alert_id,
payload.patient_id))
        elif anomalies:
            msg = "; ".join(anomalies)
            cur.execute("""
INSERT INTO incident_alerts (patient_id, alert_type, severity, message, status, created_at)
VALUES (?, 'VITAL_ANOMALY', 'Warning', ?, 'Emergency', ?)
""", (payload.patient_id, msg, now_str))
            alert_id = cur.lastrowid
            conn.commit()
        metric_data = {
            "type": "NEW_METRIC",
            "metric": {
                "id": log_id,
                "patient_id": payload.patient_id,
                "timestamp": now_str,
                "heart_rate": payload.heart_rate,
                "spo2": payload.spo2,
                "temperature": payload.temperature,
                "steps": payload.steps,
                "latitude": payload.latitude,
                "longitude": payload.longitude,
                "motion_vector": payload.motion_vector
            },
            "anomalies": anomalies,
            "is_fall": is_fall,
            "alert_id": alert_id
        }
    await manager.broadcast(metric_data)
    return {"status": "success", "metric_id": log_id, "anomalies": anomalies, "is_fall": is_fall}
@app.post("/api/alerts/{alert_id}/cancel")
async def cancel_alert(alert_id: int):
    """False Alarm Safeguard: Senior/Caregiver taps to cancel within 15-sec countdown (REQ-8)."""
    global active_countdown_task
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT status, patient_id FROM incident_alerts WHERE id = ?", (alert_id,))
        row = cur.fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Alert not found")
        if row["status"] != "Verifying_Countdown":
            return {"status": "ignored", "message": f"Alert cannot be cancelled in state '{row['status']}'"}
        cur.execute("""
UPDATE incident_alerts
SET status = 'Cancelled', doctor_notes = 'Cancelled by senior as false alarm.'
WHERE id = ?
""", (alert_id,))
        conn.commit()
        patient_id = row["patient_id"]
    if active_countdown_task and not active_countdown_task.done():
        active_countdown_task.cancel()
    await manager.broadcast({
        "type": "ALERT_CANCELLED",
        "alert_id": alert_id,
        "patient_id": patient_id,
        "message": "False alarm confirmed: emergency alert pipeline successfully aborted."
    })
    return {"status": "success", "message": "Alert cancelled successfully."}
@app.post("/api/alerts/{alert_id}/resolve")
async def resolve_alert(alert_id: int, payload: ResolveAlertPayload):
    """Doctor / Caregiver Incident Resolution & Clinical Notes Logging (Section 2.3 & 5.5)."""
    now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT status, patient_id FROM incident_alerts WHERE id = ?", (alert_id,))
        row = cur.fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Alert not found")
        if row["status"] == "Resolved":
            raise HTTPException(status_code=400, detail="Historical records for resolved incidents are
locked.")

```

```

cur.execute("""
UPDATE incident_alerts
SET status = 'Resolved', doctor_notes = ?, resolved_at = ?
WHERE id = ?
""", (payload.doctor_notes, now_str, alert_id))
conn.commit()
patient_id = row["patient_id"]
await manager.broadcast({
    "type": "ALERT_RESOLVED",
    "alert_id": alert_id,
    "patient_id": patient_id,
    "notes": payload.doctor_notes,
    "resolved_at": now_str
})
return {"status": "success", "message": "Incident resolved and securely locked."}
@app.get("/api/patients/{patient_id}/report")
def get_clinical_report(patient_id: int):
    """Generates an aggregated medical health summary report for clinical reviews and printing."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM elderly_profiles WHERE id = ?", (patient_id,))
        patient = cur.fetchone()
        if not patient:
            raise HTTPException(status_code=404, detail="Patient profile not found")
        # Telemetry aggregation
        cur.execute("""
SELECT
COUNT(*) as total_records,
AVG(heart_rate) as avg_hr,
MIN(heart_rate) as min_hr,
MAX(heart_rate) as max_hr,
AVG(spo2) as avg_spo2,
MIN(spo2) as min_spo2,
AVG(temperature) as avg_temp,
MAX(steps) as total_steps
FROM health_metrics
WHERE patient_id = ?
""", (patient_id,))
        stats = dict(cur.fetchone())
        # Incidents
        cur.execute("SELECT COUNT(*) FROM incident_alerts WHERE patient_id = ? AND
alert_type = 'FALL_DETECTED'", (patient_id,))
        falls_count = cur.fetchone()[0]
        cur.execute("SELECT COUNT(*) FROM incident_alerts WHERE patient_id = ? AND
alert_type = 'VITAL_ANOMALY'", (patient_id,))
        vital_anomalies_count = cur.fetchone()[0]
        cur.execute("SELECT COUNT(*) FROM incident_alerts WHERE patient_id = ? AND status =
'Cancelled'", (patient_id,))
        false_alarms_count = cur.fetchone()[0]
        # Recent alerts
        cur.execute("SELECT * FROM incident_alerts WHERE patient_id = ? ORDER BY id DESC
LIMIT 5", (patient_id,))
        recent_alerts = [dict(r) for r in cur.fetchall()]
        # Medications & Adherence
        cur.execute("SELECT * FROM medications WHERE patient_id = ?", (patient_id,))
        meds = [dict(r) for r in cur.fetchall()]
        total_meds = len(meds)
        taken_meds = sum(1 for m in meds if m["status"] == "taken")
        adherence_rate = round((taken_meds / total_meds * 100), 1) if total_meds > 0 else 100.0
        # Wellness
        cur.execute("SELECT * FROM daily_wellness_logs WHERE patient_id = ? ORDER BY id
DESC LIMIT 1", (patient_id,))
        wellness = cur.fetchone()
        return {
            "generated_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "patient": dict(patient),
            "stats": {
                "total_records": stats["total_records"] or 0,
                "avg_heart_rate": round(stats["avg_hr"] or 72, 1),
                "min_heart_rate": stats["min_hr"] or 70,
                "max_heart_rate": stats["max_hr"] or 85,
                "avg_spo2": round(stats["avg_spo2"] or 98.0, 1),
                "min_spo2": stats["min_spo2"] or 96.0,
                "avg_temperature": round(stats["avg_temp"] or 36.6, 1),
                "total_steps": stats["total_steps"] or 2400,
                "falls_count": falls_count,
                "vital_anomalies_count": vital_anomalies_count,
                "false_alarms_prevented": false_alarms_count,
                "medication_adherence_rate": adherence_rate
            },
            "medications": meds,
            "recent_alerts": recent_alerts,
            "wellness": dict(wellness) if wellness else {}
        }
@app.get("/api/patients/{patient_id}/export/csv")
def export_telemetry_csv(patient_id: int):
    """Exports raw time-series telemetry records as downloadable CSV."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT full_name FROM elderly_profiles WHERE id = ?", (patient_id,))
        pat = cur.fetchone()
        patient_name = pat["full_name"] if pat else f"patient_{patient_id}"
        cur.execute("""
SELECT id, timestamp, heart_rate, spo2, temperature, steps, latitude, longitude,
motion_vector
FROM health_metrics
WHERE patient_id = ?
ORDER BY id ASC
""", (patient_id,))
        rows = cur.fetchall()
        output = io.StringIO()
        writer = csv.writer(output)
        writer.writerow(["Log ID", "Patient Name", "Timestamp", "Heart Rate (bpm)", "SpO2 (%)",
"Temperature (C)", "Steps", "Latitude", "Longitude", "Motion Vector (g)"])
        for r in rows:
            writer.writerow([r["id"], patient_name, r["timestamp"], r["heart_rate"], r["spo2"],
r["temperature"], r["steps"], r["latitude"], r["longitude"], r["motion_vector"]])
        output.seek(0)
        filename = f"ecms_telemetry_{patient_name.replace(' ', '_').lower()}.csv"
        return StreamingResponse(
            io.BytesIO(output.getvalue().encode("utf-8")),
            media_type="text/csv",
            headers={"Content-Disposition": f"attachment; filename={filename}"})

```

```

)
@app.get("/api/iot/code")
def get_iot_hardware_code():
    """Provides ready-to-flash code snippets for ESP32, Raspberry Pi, and REST cURL."""
    esp32_code = """// ECMS ESP32 IoT Firmware (MAX30102 + MPU6050 + GPS NEO-6M)
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include "MAX30105.h"
#include <MPU6050_light.h>
#include <TinyGPS++.h>
const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";
const char* serverUrl = "http://192.168.1.100:8000/api/metrics"; // Set to ECMS Server IP
MAX30105 particleSensor;
MPU6050 mpu(Wire);
TinyGPSPlus gps;
void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nWiFi Connected!");
    Wire.begin(21, 22); // SDA, SCL
    particleSensor.begin(Wire, I2C_SPEED_FAST);
    particleSensor.setup();
    mpu.begin();
    mpu.calcOffsets();
}
void loop() {
    mpu.update();
    float ax = mpu.getAccX(), ay = mpu.getAccY(), az = mpu.getAccZ();
    float motionVector = sqrt(ax*ax + ay*ay + az*az); // Magnitude in g
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(serverUrl);
        http.addHeader("Content-Type", "application/json");

        String json = "{\"patient_id\\\": 1, \\\"heart_rate\\\": 74, \\\"spo2\\\": 98.0, \\\"temperature\\\": 36.6, \\\"steps\\\": 2600, \\\"latitude\\\": 11.0168, \\\"longitude\\\": 76.9558, \\\"motion_vector\\\": \" +
        String(motionVector, 2) + \"}";
        int httpResponseCode = http.POST(json);
        http.end();
    }
    delay(2500);
}"""

python_rpi_code = """# ECMS Raspberry Pi Telemetry Publisher
import requests
import time
import math
import random
SERVER_URL = "http://localhost:8000/api/metrics"
PATIENT_ID = 1
def send_telemetry():
    payload = {
        "patient_id": PATIENT_ID,
        "heart_rate": random.randint(70, 78),
        "spo2": round(random.uniform(97.0, 99.0), 1),
        "temperature": 36.6,
        "steps": 2840,
        "latitude": 11.0168 + random.uniform(-0.0001, 0.0001),
        "longitude": 76.9558 + random.uniform(-0.0001, 0.0001),
        "motion_vector": 0.98
    }
    resp = requests.post(SERVER_URL, json=payload)
    print("Ingested metric:", resp.json())
if __name__ == "__main__":
    while True:
        send_telemetry()
        time.sleep(3)
"""

curl_command = """curl -X POST "http://127.0.0.1:8000/api/metrics" \\\
-H "Content-Type: application/json" \\\
-d '{
    "patient_id": 1,
    "heart_rate": 75,
    "spo2": 98.2,
    "temperature": 36.6,
    "steps": 2650,
    "latitude": 11.0168,
    "longitude": 76.9558,
    "motion_vector": 0.98
}'"""

return {
    "esp32_arduino": esp32_code,
    "raspberry_pi_python": python_rpi_code,
    "curl_example": curl_command
}

@app.post("/api/simulate/fall")
async def trigger_simulated_fall(patient_id: int = Query(1)):
    """Wearable Simulator trigger: simulates fall motion vector (REQ-6)."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT safe_lat, safe_lng FROM threshold_configs WHERE patient_id = ?",
(patient_id,))
        thresh = cur.fetchone()
        lat = thresh["safe_lat"] if thresh else 11.0168
        lng = thresh["safe_lng"] if thresh else 76.9558
        payload = MetricPayload(
            patient_id=patient_id,
            heart_rate=102,
            spo2=95.5,
            temperature=36.7,
            steps=2540,
            latitude=lat + 0.0001,
            longitude=lng + 0.0001,
            motion_vector=3.85 # Exceeds 3.2g threshold!
        )
        return await ingest_metric(payload)

@app.post("/api/simulate/fever")
async def trigger_simulated_fever(patient_id: int = Query(1)):
    """Simulates sudden high body temperature anomaly."""
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("SELECT safe_lat, safe_lng FROM threshold_configs WHERE patient_id = ?",

```

```

(patient_id,))
        thresh = cur.fetchone()
        lat = thresh["safe_lat"] if thresh else 11.0168
        lng = thresh["safe_lng"] if thresh else 76.9558

        payload = MetricPayload(
            patient_id=patient_id,
            heart_rate=112,
            spo2=96.0,
            temperature=39.2, # High Fever!
            steps=2100,
            latitude=lat,
            longitude=lng,
            motion_vector=0.95
        )
        return await ingest_metric(payload)

@app.post("/api/simulate/reset")
async def reset_simulation(patient_id: int = Query(1)):
    """Resets patient state to normal vitals inside safe geofence."""
    global active_countdown_task
    if active_countdown_task and not active_countdown_task.done():
        active_countdown_task.cancel()
    with get_db() as conn:
        cur = conn.cursor()
        cur.execute("UPDATE incident_alerts SET status = 'Resolved' WHERE patient_id = ? AND
status IN ('Verifying_Countdown', 'Emergency')", (patient_id,))
        cur.execute("SELECT safe_lat, safe_lng FROM threshold_configs WHERE patient_id = ?",
(patient_id,))
        thresh = cur.fetchone()
        lat = thresh["safe_lat"] if thresh else 11.0168
        lng = thresh["safe_lng"] if thresh else 76.9558
        conn.commit()
        payload = MetricPayload(
            patient_id=patient_id,
            heart_rate=74,
            spo2=98.0,
            temperature=36.6,
            steps=2600,
            latitude=lat,
            longitude=lng,
            motion_vector=0.98
        )
        res = await ingest_metric(payload)
        await manager.broadcast({"type": "RESET_ALL", "patient_id": patient_id})
        return {"status": "success", "message": "State reset to normal telemetry"}

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await manager.connect(websocket)
    try:
        while True:
            # Wait for message with a 30s timeout to detect dead connections early
            data = await asyncio.wait_for(websocket.receive_text(), timeout=30.0)
            if data == "ping":
                await websocket.send_text("pong")
            except asyncio.TimeoutError:
                # No message in 30s — send a server-side ping to check if client is alive
                try:
                    await websocket.send_text("ping")
                except Exception:
                    break # Client is gone, exit loop
            except WebSocketDisconnect:
                pass # Normal client disconnect — not an error
            except Exception as e:
                print(f"[WS] Unexpected error: {e}")
            finally:
                manager.disconnect(websocket)
    if STATIC_DIR.exists():
        app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
    @app.get("/")
    def serve_index():
        index_file = STATIC_DIR / "index.html"
        if index_file.exists():
            return FileResponse(str(index_file))
        return {"message": "ECMS Backend running. Frontend static/index.html is being prepared."}
    if __name__ == "__main__":
        import uvicorn
        uvicorn.run("server:app", host="127.0.0.1", port=8000, reload=True)

```

Output:

ECMS CarePortal

Medical Personnel & Clinical Diagnostics Portal

Current View: Dr. Arvind Sharma (Doctor)

Live WS Connected

Audio Alerts: ON

Robert Jenkins

Age: 78

Status: Normal

Conditions: Hypertension, Mild Cardiac Arrhythmia, Osteoporosis

Emergency Contact: +91-9876500003 (Primary Caregiver)

Last Ingested Sync: 2026-09-29 15:14:13

Threshold Settings

HEART RATE (HR)

102 bpm

Safe Range: 55 - 105 bpm

Normal

BLOOD OXYGEN (SpO2)

95.5 %

Safe: ≥ 92%

Optimal

BODY TEMPERATURE

36.7 °C

Range: 36.0 - 37.8 °C

Normal

ACTIVITY & DECELERATION

3.85 g force

Steps: 2,540

HIGH IMPACT FALL

Real-Time Vitals Trend

Streaming time-series telemetry (Heart Rate & SpO2)

LIVE STREAM

Live GPS & Geo-Fence Tracker

Monitors senior within authorized safe radius

Safe in Zone

WEARABLE SENSOR SIMULATOR

Simulate live patient telemetry to test SRS triggers:

Start Auto Stream

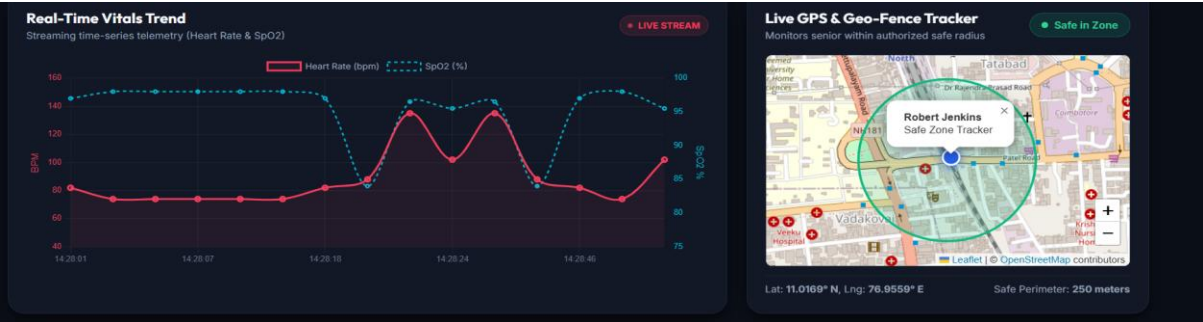
Simulate Fall Impact (>3.2g)

Tachycardia (135 bpm)

Hypoxemia (84% SpO2)

Wander Out of Zone

Reset All to Normal



ID	Alert Type	Severity	Message	Status	Doctor Resolution	Action
#18	FALL_DETECTED	Warning	High deceleration impact detected! 15s verification countdown started.	Cancelled	Cancelled by senior as false alarm.	—
#17	VITAL_ANOMALY	Warning	Geo-fence breach: Senior is 779.1m away from safe center (Limit: 250.0m)	Resolved	Pending review	Locked
#16	VITAL_ANOMALY	Warning	Hypoxemia warning: SpO2 level 84.0% dropped below safe limit 92.0%	Resolved	Pending review	Locked
#15	VITAL_ANOMALY	Warning	Tachycardia detected: Heart rate 135 bpm exceeds maximum 105 bpm	Resolved	Pending review	Locked
#14	FALL_DETECTED	Warning	High deceleration impact detected! 15s verification countdown started.	Cancelled	Cancelled by senior as false alarm.	—

WEARABLE SENSOR SIMULATOR

Simulate live patient telemetry to test SRS triggers:

Start Auto Stream

Simulate Fall Impact (>3.2g)

Tachycardia (135 bpm)

Hypoxemia (84% SpO2)

Wander Out of Zone

Reset All to Normal

[PUSH] → Caregiver Mobile App Token #84920

2026-09-23 12:29:16

EMERGENCY: Fall confirmed for Robert Jenkins

[EMAIL] → family.caregiver@ecms-cit.edu

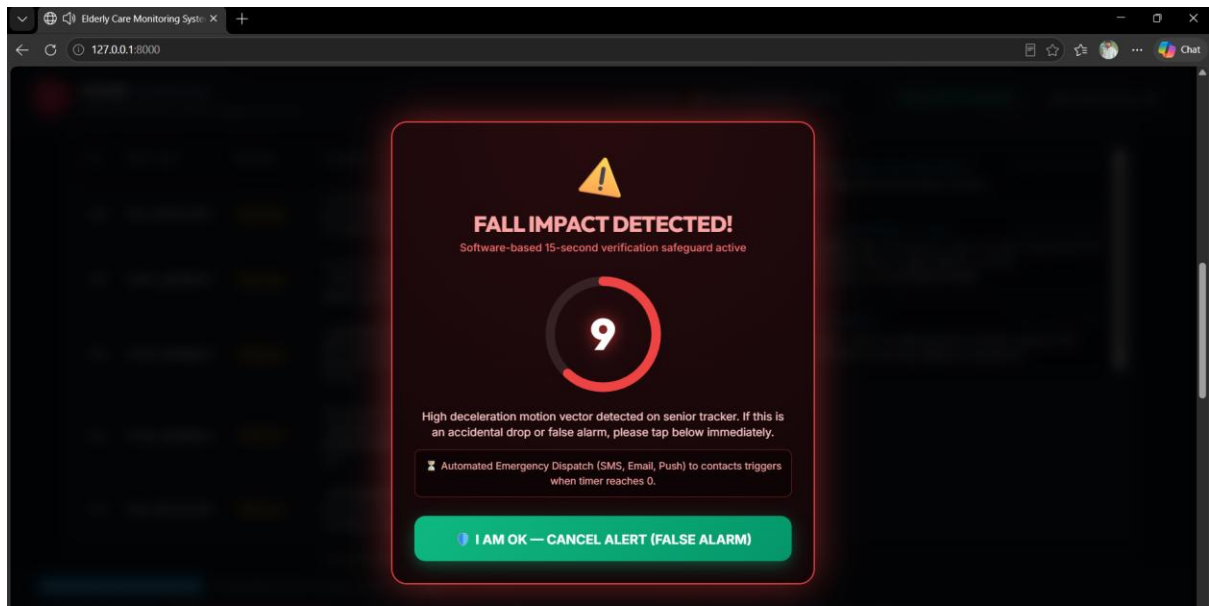
2026-09-23 12:29:16

Subject: URGENT Medical Emergency - Robert Jenkins High impact fall detected. Vitals: HR=98 bpm, SpO2=96.0%. Location: https://maps.google.com/?q=11.0169,76.9559

[SMS] → +91-9876500003

2026-09-23 12:29:16

CRITICAL ALERT: Confirmed fall detected for Robert Jenkins! GPS: (11.01690, 76.95590). Immediate assistance dispatched.



RESULT:

Thus, the implementation of the Software Requirements Specification (SRS) document for the software-based **Elderly Care Monitoring System** has been completed and verified according to project guidelines.